МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по производственной практике

ПП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ Аксенов Максим Витальевич
(подпись) (фамилия, имя, отчество)

Группы ИПо-42

Руководитель практической подготовки от профильной организации

_____Декан математического факультета_____
(должность)

_____ Скорнякова Анна Юрьевна
(подпись) (фамилия, имя, отчество)

МП «16» марта 2026 года

Руководитель практической подготовки от института

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

«16» марта 2026 года

Пермь, 2026 год

ОГЛАВЛЕНИЕ

Введение 3
Раздел 1. Организационно-подготовительный этап прохождения практики на предприятии 5
Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии 5
Раздел 2. Ознакомление со структурой и характером деятельности подразделения 8
Тема 2.1. Ознакомление с организацией работы на предприятии или в структурном подразделении 8
Тема 2.2. Ознакомление с должностными и функциональными обязанностями 11
Раздел 3. Работа на рабочих местах или в подразделениях предприятия 15
Тема 3.1. Выполнение производственных заданий 15
Заключение 53
Список используемых источников 54

ВВЕДЕНИЕ

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Производственная практика направлена на освоение обучающимися профессиональных компетенций в процессе самостоятельного выполнения определенных видов работ, необходимых для осуществления профессиональной деятельности в максимально приближенных к производству условиях, в рамках модулей ППССЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем.

В настоящей работе представлен отчет о прохождении производственной практики студента-практиканта. Основными видами деятельности которой являлись: разработка требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент; моделирование процесса разработки программного обеспечения.

Цель практики: закрепление, углубление теоретических знаний, полученных в техникуме, и приобретение практического опыта в области внедрения и поддержки программного обеспечения.

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

1. Выполнять интеграцию модулей в программное обеспечение;
2. Выполнять отладку программного модуля с использованием специализированных программных средств;
3. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения;
4. Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;
5. Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;
6. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности

7. Планировать и реализовывать собственное профессиональное и личностное развитие;

8. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами

9. Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста.

10. Использовать информационные технологии в профессиональной деятельности

11. Пользоваться профессиональной документацией на государственном и иностранном языках.

Место прохождения практики: Федеральное государственное бюджетное образовательное учреждение высшего образования "Пермский государственный гуманитарно-педагогический университет", математический факультет.

Сроки прохождения практики: 09.03.2026-17.03.2026.

РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ

Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии

Общие функциональные обязанности сотрудника ИТ-подразделения:

1. Администрирование и настройка оборудования (Настраивать и обслуживать компьютерное и периферийное оборудование, сети, серверы и программное обеспечение. Вносить изменения в конфигурацию IT-инфраструктуры в пределах своих полномочий.)
2. Установка и обновление программного обеспечения (Устанавливать, настраивать и обновлять программные средства, связанные с учебной, научной деятельностью и административными задачами. Обеспечивать безопасность и защиту программного обеспечения.)
3. Обеспечение информационной безопасности (Контролировать соблюдение правил безопасности информации.)
4. Обучение и консультирование пользователей (Оказывать консультации сотрудникам и студентам по вопросам работы с ИТ-ресурсами. Обучать пользователей безопасному использованию оборудования и программного обеспечения.)
5. Взаимодействие с внешними поставщиками (Осуществлять коммуникацию с поставщиками программных и аппаратных решений.)
6. Предоставление отчетности (Готовить отчеты о выполненной работе, инвентаризации оборудования и использовании ресурсов по запросу руководства.)
7. Разработка, тестирование, сопровождение и поддержка созданного программного обеспечения.

Правила техники безопасности сотрудника ИТ-подразделения.

1. Перед началом работы:

- Рационально организовать рабочее место. Если в помещении расположены несколько персональных компьютеров, то для обеспечения безопасности расстояние между ними должно быть не менее 1,5 м.

- Проверить исправность персонального компьютера и подготовить его к работе.

- Отрегулировать освещение на рабочем месте, убедиться в достаточности освещённости, отсутствии отражений на экране, отсутствии встречного светового потока.

- Проверить правильность положения оборудования, угла наклона экрана, положение клавиатуры и расположение элементов компьютера в соответствии с требованиями эргономики.

2. Во время работы:

- Выполнять только ту работу, которая была поручена и по которой был проведён инструктаж.

- В течение всего рабочего дня содержать в порядке и чистоте рабочее место.

- Соблюдать санитарные нормы и режимы работы и отдыха.

- Соблюдать правила эксплуатации вычислительной техники в соответствии с инструкциями по эксплуатации.

- Соблюдать установленные режимом рабочего времени регламентированные перерывы в работе и выполнять упражнения для глаз, шеи, рук, туловища, ног.

3. В аварийных ситуациях:

- При обнаружении неполадок в работе персонального компьютера прекратить работу, выключить компьютер и сообщить об этом непосредственному руководителю для организации ремонта.

- Не следует самому устранять технические неполадки персонального компьютера.

- Не производить работу при снятом корпусе компьютера.

- При несчастном случае, отравлении, внезапном заболевании необходимо немедленно оказать первую помощь пострадавшему, вызвать врача или помочь доставить пострадавшего к врачу, а затем сообщить руководителю о случившемся.

- При обнаружении пожара или признаков горения (задымление, запах гари, повышение температуры и т. п.) необходимо немедленно уведомить об этом пожарную охрану по телефону 01.

Правила техники безопасности при работе с электрическими приборами и устройствами:

1. Перед началом работы необходимо осмотреть оборудование и убедиться в его исправности, в том числе проверить электрический шнур, вилку и розетку.

2. Не допускать натягивания, скручивания, перегиба и пережима шнуров электропитания, проводов и кабелей, а также нахождения на них каких-либо предметов и соприкосновения их с нагретыми поверхностями.

3. Не допускать попадания влаги на поверхности ПЭВМ, периферийных устройств и другого оборудования. Не протирать влажной или мокрой ветошью оборудование, которое находится под электрическим напряжением.

4. Не прикасаться к движущимся частям средств оргтехники и другого оборудования.

5. Не работать при снятых и повреждённых защитных ограждениях средств оргтехники и другого оборудования.

6. Не касаться элементов средств оргтехники и другого оборудования влажными руками.

7. Не переключать интерфейсные кабели, вскрывать корпуса средств оргтехники и другого оборудования и самостоятельно производить их ремонт.

8. Не использовать самодельные электроприборы и электроприборы, не имеющие отношения к выполнению производственных обязанностей.

РАЗДЕЛ 2. ОЗНАКОМЛЕНИЕ СО СТРУКТУРОЙ И ХАРАКТЕРОМ ДЕЯТЕЛЬНОСТИ ПОДРАЗДЕЛЕНИЯ

Тема 2.1. Ознакомление с организацией работы на предприятии или в структурном подразделении

Федеральное государственное бюджетное образовательное учреждение высшего образования «Пермский государственный гуманитарно-педагогический университет» (ПГГПУ) - высшее учебное заведение России, расположен в Перми. Основан в 1921 году. До 2012 года - Пермский государственный педагогический университет (ПГПУ).

Режим работы: понедельник-пятница 9:00-18:00.

Правила внутреннего распорядка для сотрудников ФГБОУ ВПО «Пермский государственный гуманитарно-педагогический университет» утверждены Протоколом заседания ученого Совета №1 от 17.09.2013г. и регламентируют порядок приема и увольнения работников, основные права, обязанности и ответственность сторон трудовых отношений (работников и работодателя или уполномоченных им лиц), режим работы, время отдыха, применяемые к работникам меры поощрения и взыскания, правила поддержания учебного порядка, а также порядка в помещениях и на территории Университета.

В ПГГПУ широко применяются как индивидуальные, так и коллективные формы организации труда (в зависимости от вида выполняемых работ).

По состоянию на 2025 год в структуру учебного заведения входят 13 факультетов, 29 кафедр, школа, НИИ, центры, лаборатории, фундаментальная библиотека, пресс-служба. С 2002 года существуют филиалы университета в Осе, Лысьве, Кунгуре и Чайковском, а также представительство в Кудымкаре.

Согласно штатному расписанию, в 2025 году в Университете работает более 500 преподавателей и научных сотрудников.

С 2019 года ректором университета является кандидат исторических наук, доцент Константин Борисович Егоров.

Факультеты ПГГПУ:

- Математический факультет
- Филологический факультет
- Факультет физической культуры
- Факультет педагогики и методики начального образования
- Факультет педагогики и психологии детства

- Естественнонаучный факультет
- Факультет иностранных языков
- Физический факультет
- Исторический факультет
- Факультет музыки
- Факультет психологии
- Факультет информатики и экономики
- Факультет правового и социально-педагогического образования

Математический факультет появился в 1975 году в результате разделения физико-математического факультета на математический и физический. На факультете работает 15 преподавателей, из них: 1 доктор наук, 8 кандидатов наук. Структура факультета утверждается Ученым советом Университета. В состав факультета входят:

- ✓ ученый совет факультета,
- ✓ деканат,
- ✓ кафедра высшей математики и методики обучения математике,
- ✓ научно-методическая лаборатория «Методическое сопровождение обучения математике в основной общеобразовательной школе»,
- ✓ учебно-методический кабинет,
- ✓ компьютерный класс,
- ✓ кабинет истории математики и математического образования,
- ✓ учебные кабинеты,
- ✓ иные подразделения (академические группы обучающихся, органы студенческого самоуправления).

Миссия математического факультета – организация качественной подготовки будущих учителей математики и информатики, способных осуществлять профессиональную педагогическую, организационную, культурно-массовую деятельность.

Цель работы лаборатории – способствовать взаимодействию учителей математики Пермского края по актуальным проблемам современного образования, поддерживать приобщение студентов к будущей профессиональной деятельности.

В ПГГПУ в качестве отдельного структурного подразделения функционирует Управление информатизации, включающее Отдел разработки и внедрения программных решений и Отдел технической поддержки и системного администрирования. Целью

работы Управления является построение интегрированной системы управления для всех процессов в Университете на базе информационных технологий.

Принципы управления факультетами в ПГГПУ закреплены в Положениях о факультетах, а также регулируются федеральным законодательством. Они характеризуются соответствием современным стандартам эффективности, прозрачности, ориентированности на образовательные и научные цели, основаны на сочетании демократии, инновационности, что способствует развитию Университета как современного образовательного и научного центра, отвечающего требованиям времени и интересам Общества.

Осуществление должностных обязанностей в ПГГПУ включает структурированный и ответственный подход к выполнению своих профессиональных функций, соответствующий должностной инструкции и внутренним нормативным актам университета.

Характеристика Пермского государственного гуманитарно-педагогического университета:

1. Основание: Год основания 1921.
2. Форма собственности: Государственная.
3. Тип университета: Гуманитарно-педагогический.
4. Количество студентов: Более 5 600 обучающихся.
5. Количество преподавателей: Более 500 членов профессорско-преподавательского состава и научных сотрудников. Среди них более 250 кандидатов наук и около 80 докторов наук.
6. Фонд нормативной и оборотной учебной литературы, оснащение аудитории и лабораторий: Формирование фонда осуществляется путём приобретения документов в соответствии с образовательными программами, учебными планами, программами курсов и тематикой научных исследований. В состав фонда входит основной книжный фонд на русском и иностранном языках (учебная, учебно-методическая, научная и художественная литература), периодические издания на русском и иностранных языках, электронные издания, фонд редких изданий. Библиотека активно сотрудничает с цифровыми библиотеками, библиотеками крупных университетов и научных учреждений. Состав учебных помещений определяется в соответствии с учебными планами и программами, исходя из условий проведения занятий в одну смену.

Показатели деятельности ПГГПУ в 2025 году:

1. Распределение студентов по формам обучения: Бакалавриат – 70%, магистратура – 20%, аспирантура – 10%.

2. Объем научной деятельности: Более 1000 публикаций, участие в международных проектах, рост грантов.

3. Международное сотрудничество: Более 30 зарубежных партнеров, расширение обменных программ.

4. Инновационные проекты: Внедрение цифровых образовательных ресурсов, разработка научно-педагогических инноваций.

5. Приём студентов: В 2025 году ПГГПУ зачислено студентов на 1036 бюджетных мест и 456 платных. Минимальный проходной балл составил 133 на направление «Педагогическое образование с двумя профилями подготовки: химия/география», максимальный балл - 224 на направление «Педагогическое образование с двумя профилями подготовки: музыкальное и дополнительное образование».

6. Рейтинг: по данным локального рейтинга вузов Приволжского федерального округа, проводимого аналитическим агентством RAEX, в 2025 году ПГГПУ находится на 26-й позиции. В национальном сводном рейтинге «Интерфакс» учебное заведение заняло 299–301-е место.

Тема 2.2. Ознакомление с должностными и функциональными обязанностями

Согласно Правилам внутреннего распорядка ФГБОУ ВПО «Пермский государственный гуманитарно-педагогический университет», каждый сотрудник имеет право на:

- заключение, изменение и расторжение трудового договора в порядке и на условиях, которые установлены Трудовым кодексом РФ, иными федеральными законами;
- рабочее место, соответствующее государственным нормативным требованиям охраны труда и условиям, предусмотренным Коллективным договором;
- своевременную и в полном объеме выплату заработной платы в соответствии со своей квалификацией, сложностью труда, количеством и качеством выполненной работы;
- отдых, обеспечиваемый установлением нормальной продолжительности рабочего времени, сокращенного рабочего времени для отдельных профессий и категорий работников, предоставлением еженедельных выходных дней, нерабочих праздничных дней, оплачиваемых ежегодных отпусков;
- полную достоверную информацию об условиях труда и требованиях охраны труда на рабочем месте;
- профессиональную подготовку, переподготовку и повышение своей квалификации в порядке, установленном Трудовым кодексом РФ, иными федеральными законами;
- на защиту своей профессиональной чести и достоинства;

- на материально-техническое и организационное обеспечение своей профессиональной и научной деятельности;

- избирать и быть избранными в органы управления Университета;

- участвовать в обсуждении и решении важнейших вопросов учебной, научной, творческой и производственной деятельности Университета и его структурных подразделений, в том числе на основе членства в общественных организациях и в составе органов управления Университетом;

- пользоваться бесплатно в соответствии с установленным Уставом Университета порядком информационными фондами, услугам и учебных, научных, социально-бытовых, лечебных и других подразделений Университета;

- выбирать методы и средства обучения, наиболее отвечающие их индивидуальным особенностям и обеспечивающие высокое качество учебного процесса.

Все работники Университета обязаны:

а) работать честно и добросовестно, строго выполнять условия трудового договора и должностные обязанности, поддерживать и укреплять высокий имидж Университета;

б) строго соблюдать положения Устава ПГГПУ, Правил внутреннего распорядка Университета, Коллективного договора, Правил пожарной безопасности и других локальных нормативных актов;

в) соблюдать трудовую дисциплину, своевременно и точно исполнять распоряжения администрации, использовать все рабочее время для производительного труда, воздерживаться от действий, мешающих другим работникам выполнять их трудовые обязанности;

г) соблюдать требования по охране труда, технике безопасности, пожарной безопасности, производственной санитарии, гигиене труда, предусмотренные правилами и инструкциями; работать в соответствии с профессиональной принадлежностью в выданной спецодежде, спецобуви; пользоваться необходимыми средствами индивидуальной защиты;

д) соблюдать установленный порядок хранения материальных ценностей и документов; е) содержать в порядке и чистоте свое рабочее место, соблюдать чистоту на территории Университета;

ж) беречь имущество Университета (в том числе имущество третьих лиц, за которое Университет несет ответственность), эффективно и по назначению использовать оборудование, бережно относиться к инструментам, измерительным приборам, спецодежде, другим предметам, выдаваемым в пользование работникам, экономно и

рационально расходовать сырье, материалы, энергию, топливо и другие материальные ресурсы.

Должностная инструкция системного администратора Пермского государственного гуманитарно-педагогического университета регламентирует обязанности, права, ответственность сотрудника и требования к кандидатам на эту должность. Основные положения должностной инструкции:

- Системный администратор относится к категории технических специалистов.

- В своей деятельности руководствуется законодательством РФ, уставом Университета, внутренними нормативными актами, данной инструкцией и заданиями руководства.

- Цель должности: обеспечивать стабильную работу информационных систем Университета, их безопасность, развитие и поддержку, а также автоматизацию учебных и административных процессов.

- Задачи и обязанности: разработка, сопровождение и обновление программных средств, используемых в Университете, включая учебные и административные системы. Обеспечение функционирования информационных систем и баз данных учреждения. Настройка и оптимизация работы серверов, компьютеров и программного обеспечения. Участие в технической поддержке сотрудников и студентов по вопросам информационных технологий. Создание и внедрение новых программных решений, автоматизация бизнес-процессов Университета. Обеспечение безопасности информационных систем, внедрение мер по защите данных. Участие в учебных, научных и административных проектах университета, связанных с IT и др.

- Ответственность: за сохранность и безопасность информационных систем и данных. За соблюдение правил эксплуатации программного обеспечения и технического оборудования. За нарушение требований законодательства, нормативных актов и внутренней политики университета в области информационных технологий.

Сотрудник ИТ-подразделения ПГГПУ имеет право на:

1. Доступ к информационно-техническим ресурсам (Получать доступ к серверным системам, сетевым ресурсам и внутренним базам данных факультета и университета. Иметь право на создание, изменение и удаление учетных записей пользователей и оборудования.)

2. Право на поддержку и обучение (Право на получение необходимого обучения и повышения квалификации для эффективного выполнения своих обязанностей. Право на доступ к технической документации и инструкциям.)

3. Право на обеспечение безопасности (Право предлагать и внедрять меры по обеспечению безопасности ИТ-инфраструктуры. Право отслеживать и реагировать на попытки несанкционированного доступа или угрозы информационной безопасности.)

4. Право конфиденциальности личных данных (Право на защиту своих личных данных и соблюдение конфиденциальности в работе с пользовательскими данными.)

5. Право участвовать в принятии решений (Право вносить предложения по улучшению ИТ-инфраструктуры и технологий. Право участвовать в обсуждении IT-политики.)

РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ

Тема 3.1. Выполнение производственных заданий

3.1.1. Использование выбранной системы контроля версий

В процессе проекта необходимо создать удобное и функциональное приложение, позволяющее менеджерам осуществлять управление данными о партнерах, выполнять добавление, редактирование и удаление информации для повышения эффективности работы. А также реализовать необходимые интерфейсы для пользователей системы и обработку исключительских ситуаций в приложении.

Системы контроля версий позволяют отслеживать изменения в файлах, хранить несколько версий одного и того же документа и возвращаться к более ранним версиям.

В проекте была использована система контроля версий Git, которая позволяет отслеживать и записывать любые изменения в наборе файлов и возвращаться к предыдущим версиям проекта.

Особенности ее работы**:**

- У каждого разработчика есть полноценная копия репозитория, включая всю историю проекта.
- Каждый раз, когда делается коммит, Git сохраняет текущее состояние всех файлов, как миниатюрную файловую систему. Если файл не менялся, Git не копирует его заново, а делает ссылку на уже сохранённую версию.
- Позволяет создавать параллельные ветки разработки, где можно экспериментировать и вносить изменения, не затрагивая основную версию. После проверки тестовые ветки легко объединяются с основной.
- Система помогает обнаружить и разрешить конфликты изменений.
- Каждый коммит связывается с конкретным автором, что повышает прозрачность и ответственность в команде.

Принцип работы. Основная идея системы — сохранить полный «поток снимков» проекта, чтобы можно было:

–отслеживать историю изменений и быстро находить, кто и что изменил;

восстанавливать предыдущие версии файлов, если в новой версии обнаружились ошибки или проблемы;

–безопасно экспериментировать, создавая ветки проекта, не боясь повредить основную версию;

–объединять изменения от нескольких участников без путаницы и конфликтов.

Главная особенность Git — это подход к хранению данных. Вместо простого списка изменений Git сохраняет снимки состояния файлов. Каждый раз, когда вы делаете коммит, система «фотографирует» текущее состояние всех отслеживаемых файлов и сохраняет эту информацию. Если в каком-то файле ничего не изменилось, Git просто делает ссылку на уже сохраненную версию.

Каждый коммит защищен уникальной контрольной суммой, что позволяет проверить целостность истории и убедиться, что никто ничего не удалил и не подменил код задним числом.

С момента появления Git стал самой популярной системой контроля версий. Он работает быстро, не зависит от центрального сервера, легко масштабируется на проекты любого размера и предоставляет мощные инструменты для анализа истории. Git — это специальная база данных, которая хранит не только все файлы проекта, но и полную историю их изменений, а также всю служебную информацию, необходимую для работы. Физически эта «база данных» представляет собой скрытую папку .git в корне проекта.

3.1.2. Разработка программного обеспечения

В проекте использовалась итерационная модель разработки, т.к. она предполагает поэтапное создание продукта, позволяет оперативно выявлять риски и контролировать качество создаваемой программы.

Для создания приложения использовались такие инструменты, как: интегрированная среда разработки MS Visual Studio и табличный процессор Microsoft Office Excel.

Задание выполнено в среде разработки MS Visual Studio с использованием библиотек .NET Framework, Microsoft.Win32 и OfficeOpenXml.

Диаграмма классов представлена на рисунке 1.

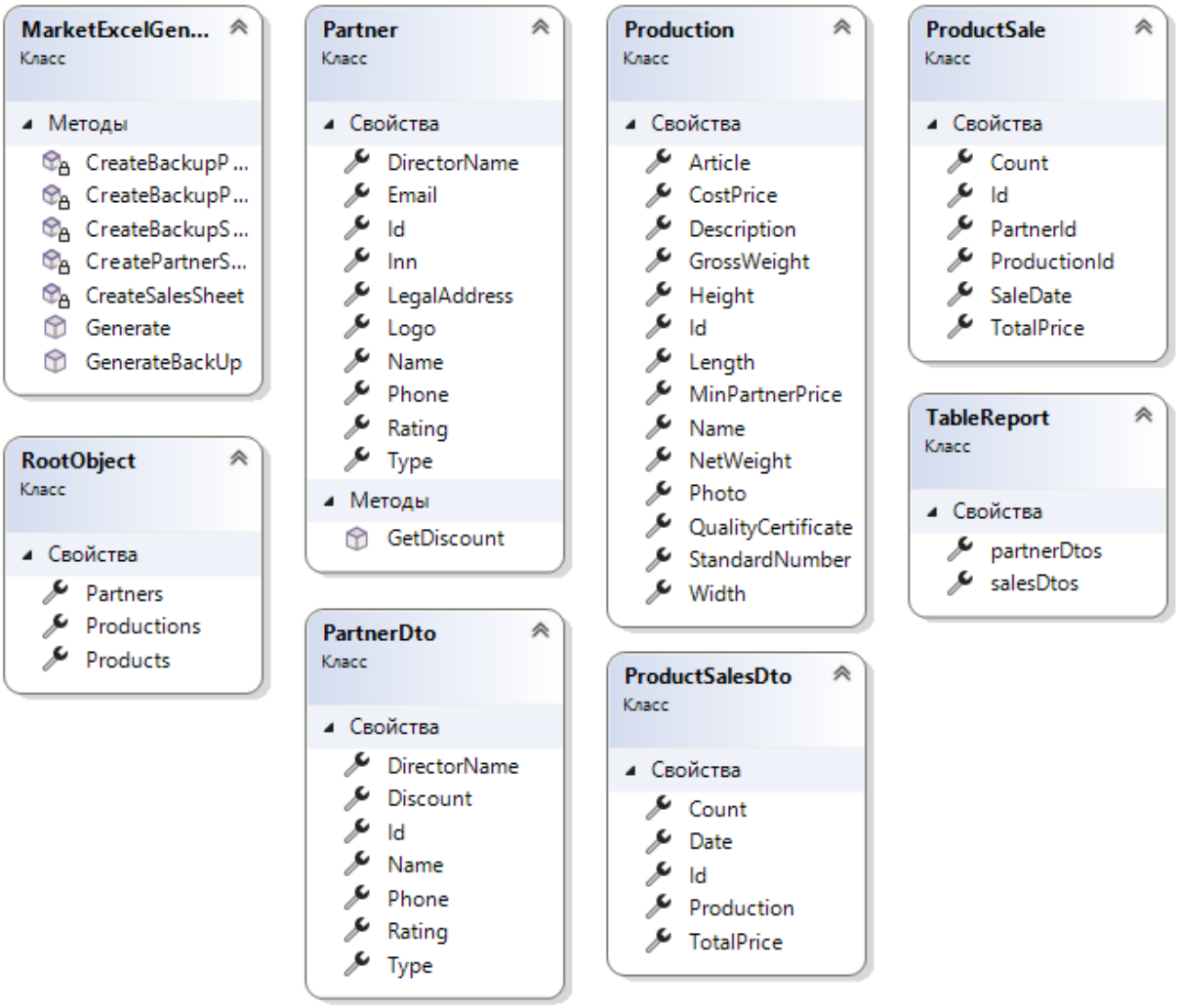

Рисунок 1 - Диаграмма классов

Код программного модуля согласно требованиям:

```
using System;
using System.Collections.Generic;
using OfficeOpenXml;
using ClassLibrary.DTOs;

namespace ClassLibrary.Classes
{
    public class MarketExcelGenerator
    {
        public static byte[] Generate(TableReport tableReport, int isInOneFile)
        {
            ExcelPackage.License.SetNonCommercialPersonal("Aksenov");
            var package = new ExcelPackage();

            switch (isInOneFile)
            {
                case 0:
                case 1:
                    // Создаем таблицу для партнеров
                    CreatePartnerSheet(package, tableReport.partnerDtos);
                    // Создаем таблицу для истории продаж, если нужен
                    if (isInOneFile == 0 || isInOneFile == 2)
                    {
                        CreateSalesSheet(package, tableReport.salesDtos);
                    }
                    break;
```

```
            case 2:
                CreatePartnerSheet(package, tableReport.partnerDtos);
                CreateSalesSheet(package, tableReport.salesDtos);
                break;
        }

        return package.GetAsByteArray();
    }

    // Метод для создания таблицы партнеров
    private static void CreatePartnerSheet(ExcelPackage package, List<PartnerDto> partnerDTOs)
    {
        var worksheet = package.Workbook.Worksheets.Add("Партнеры");
        // Заголовки таблицы
        worksheet.Cells["B2"].Value = "Тип";
        worksheet.Cells["C2"].Value = "Наименование";
        worksheet.Cells["D2"].Value = "Скидка";
        worksheet.Cells["E2"].Value = "ФИО директора";
        worksheet.Cells["F2"].Value = "Телефон";
        worksheet.Cells["G2"].Value = "Рейтинг";

        int row = 3;
        foreach (var partner in partnerDTOs)
        {
            worksheet.Cells[$"B{row}"].Value = partner.Type;
            worksheet.Cells[$"C{row}"].Value = partner.Name;
            worksheet.Cells[$"D{row}"].Value = partner.Discount;
            worksheet.Cells[$"E{row}"].Value = partner.DirectorName;
            worksheet.Cells[$"F{row}"].Value = partner.Phone;
            worksheet.Cells[$"G{row}"].Value = partner.Rating;
            row++;
        }
    }

    // Метод для создания таблицы истории продаж
    private static void CreateSalesSheet(ExcelPackage package, List<ProductSalesDto> salesDTOs)
    {
        var worksheet = package.Workbook.Worksheets.Add("История закупок");
        // Заголовки таблицы
        worksheet.Cells["B2"].Value = "Дата";
        worksheet.Cells["C2"].Value = "Количество";
        worksheet.Cells["D2"].Value = "Продукция";
        worksheet.Cells["E2"].Value = "Итоговая сумма";

        int row = 3;
        foreach (var sale in salesDTOs)
        {
            worksheet.Cells[$"B{row}"].Value = sale.Date;
            worksheet.Cells[$"C{row}"].Value = sale.Count;
            worksheet.Cells[$"D{row}"].Value = sale.Production;
            worksheet.Cells[$"E{row}"].Value = sale.TotalPrice;
            row++;
        }
    }

    public static byte[] GenerateBackUp(RootObject rootObject)
    {
        ExcelPackage.License.SetNonCommercialPersonal("Aksenov");
        var package = new ExcelPackage();

        // Создание таблицы для партнеров
        CreateBackupPartnerSheet(package, rootObject.Partners);
        // Создание таблицы для производств
        CreateBackupProductionSheet(package, rootObject.Productions);
        // Создание таблицы для продаж товаров
        CreateBackupSalesSheet(package, rootObject.Products);

        return package.GetAsByteArray();
    }

    private static void CreateBackupPartnerSheet(ExcelPackage package, List<Partner> partners)
    {
        var sheet = package.Workbook.Worksheets.Add("Партнеры");
        sheet.Cells["B2"].Value = "Id";
        sheet.Cells["C2"].Value = "Тип";
        sheet.Cells["D2"].Value = "Наименование";
        sheet.Cells["E2"].Value = "Юридический адрес";
```

```
            sheet.Cells["F2"].Value = "ИНН";
            sheet.Cells["G2"].Value = "ФИО директора";
            sheet.Cells["H2"].Value = "Телефон";
            sheet.Cells["I2"].Value = "Email";
            sheet.Cells["J2"].Value = "Логотип";
            sheet.Cells["K2"].Value = "Рейтинг";

            int row = 3;
            foreach (var partner in partners)
            {
                sheet.Cells[$"B{row}"].Value = partner.Id;
                sheet.Cells[$"C{row}"].Value = partner.Type;
                sheet.Cells[$"D{row}"].Value = partner.Name;
                sheet.Cells[$"E{row}"].Value = partner.LegalAddress;
                sheet.Cells[$"F{row}"].Value = partner.Inn;
                sheet.Cells[$"G{row}"].Value = partner.DirectorName;
                sheet.Cells[$"H{row}"].Value = partner.Phone;
                sheet.Cells[$"I{row}"].Value = partner.Email;
                sheet.Cells[$"J{row}"].Value = partner.Logo;
                sheet.Cells[$"K{row}"].Value = partner.Rating;
                row++;
            }
        }

        private static void CreateBackupProductionSheet(ExcelPackage package, List<Production> pro-
ductions)
        {
            var sheet = package.Workbook.Worksheets.Add("Производство");
            sheet.Cells["B2"].Value = "Id";
            sheet.Cells["C2"].Value = "Артикул";
            sheet.Cells["D2"].Value = "Наименование";
            sheet.Cells["E2"].Value = "Описание";
            sheet.Cells["F2"].Value = "Фото";
            sheet.Cells["G2"].Value = "Мини цена партнера";
            sheet.Cells["H2"].Value = "Длина";
            sheet.Cells["I2"].Value = "Ширина";
            sheet.Cells["J2"].Value = "Высота";
            sheet.Cells["K2"].Value = "Вес.net";
            sheet.Cells["L2"].Value = "Вес.брутт";
            sheet.Cells["M2"].Value = "Сертификат качества";
            sheet.Cells["N2"].Value = "Стандартный номер";
            sheet.Cells["O2"].Value = "Цена себестоимости";

            int row = 3;
            foreach (var prod in productions)
            {
                sheet.Cells[$"B{row}"].Value = prod.Id;
                sheet.Cells[$"C{row}"].Value = prod.Article;
                sheet.Cells[$"D{row}"].Value = prod.Name;
                sheet.Cells[$"E{row}"].Value = prod.Description;
                sheet.Cells[$"F{row}"].Value = prod.Photo;
                sheet.Cells[$"G{row}"].Value = prod.MinPartnerPrice;
                sheet.Cells[$"H{row}"].Value = prod.Length;
                sheet.Cells[$"I{row}"].Value = prod.Width;
                sheet.Cells[$"J{row}"].Value = prod.Height;
                sheet.Cells[$"K{row}"].Value = prod.NetWeight;
                sheet.Cells[$"L{row}"].Value = prod.GrossWeight;
                sheet.Cells[$"M{row}"].Value = prod.QualityCertificate;
                sheet.Cells[$"N{row}"].Value = prod.StandardNumber;
                sheet.Cells[$"O{row}"].Value = prod.CostPrice;
                row++;
            }
        }

        private static void CreateBackupSalesSheet(ExcelPackage package, List<ProductSale> sales)
        {
            var sheet = package.Workbook.Worksheets.Add("Продажи товаров");
            sheet.Cells["B2"].Value = "Id";
            sheet.Cells["C2"].Value = "Partner Id";
            sheet.Cells["D2"].Value = "Production Id";
            sheet.Cells["E2"].Value = "Кол-во";
            sheet.Cells["F2"].Value = "Дата";
            sheet.Cells["G2"].Value = "Общая цена";

            int row = 3;
            foreach (var sale in sales)
            {
```

```
                sheet.Cells[$"B{row}"].Value = sale.Id;
                sheet.Cells[$"C{row}"].Value = sale.PartnerId;
                sheet.Cells[$"D{row}"].Value = sale.ProductionId;
                sheet.Cells[$"E{row}"].Value = sale.Count;
                sheet.Cells[$"F{row}"].Value = sale.SaleDate.ToString(); // Можно оставить так или
применить форматирование
                sheet.Cells[$"G{row}"].Value = sale.TotalPrice;
                row++;
            }
        }
    }

    // Остальные классы данных - без изменений, их структура выглядит логичной.
    public class RootObject
    {
        public List<Partner> Partners { get; set; }
        public List<Production> Productions { get; set; }
        public List<ProductSale> Products { get; set; }
    }

    public class TableReport
    {
        public List<PartnerDto> partnerDtos { get; set; }
        public List<ProductSalesDto> salesDtos { get; set; }
    }
}


using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using System.Runtime.Serialization;

namespace ClassLibrary.Classes
{
    [DataContract]
    [Table("partners", Schema = "app")]
    public class Partner
    {
        [Key]
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
        [DataMember]
        [Column("id")]
        public int Id { get; set; }

        [Required]
        [DataMember]
        [Column("type")]
        public string Type { get; set; }

        [Required]
        [DataMember]
        [Column("name")]
        public string Name { get; set; }

        [Required]
        [DataMember]
        [Column("legal_address")]
        public string LegalAddress { get; set; }

        [Required]
        [DataMember]
        [Column("inn")]
        public string Inn { get; set; }  // Именование без заглавных букв, как принято

        [DataMember]
        [Column("director_name")]
        public string DirectorName { get; set; }

        [DataMember]
        [Column("phone")]
        public string Phone { get; set; }

        [DataMember]
        [Column("email")]
        public string Email { get; set; }
```

```
        [DataMember]
        [Column("logo")]
        public string? Logo { get; set; }

        [DataMember]
        [Column("rating")]
        public int Rating { get; set; }

        public int GetDiscount(List<ProductSale> productSales)
        {
            double totalproducts = 0;
            List<ProductSale> PartnerProductSales = productSales.FindAll(x => x.PartnerId == Id);
            foreach (ProductSale PartnerProductSalesItem in PartnerProductSales)
            {
                totalproducts += PartnerProductSalesItem.Count;
            }
            bool Discount0 = totalproducts < 10000;
            bool Discount5 = totalproducts > 10000 && totalproducts < 50000;
            bool Discount10 = totalproducts > 50000 && totalproducts < 300000;
            bool Discount15 = totalproducts > 300000;
            if (Discount0)
                return 0;
            else if (Discount5)
                return 5;
            else if (Discount10)
                return 10;
            else
                return 15;
        }
    }
}


using System;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using System.Runtime.Serialization;

namespace ClassLibrary.Classes
{
    [DataContract]
    [Table("production", Schema = "app")] // указание схемы, если нужно
    public class Production
    {
        [DataMember]
        [Key]
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
        [Column("id")]
        public int Id { get; set; }

        [DataMember]
        [Required]
        [Column("article")]
        public string Article { get; set; }

        [DataMember]
        [Required]
        [Column("name")]
        public string Name { get; set; }

        [DataMember]
        [Column("description")]
        public string Description { get; set; }

        [DataMember]
        [Column("photo")]
        public string Photo { get; set; }

        [DataMember]
        [Required]
        [Column("min_partner_price")]
        public double MinPartnerPrice { get; set; }

        [DataMember]
        [Column("length")]
        public int? Length { get; set; }
```

```
        [DataMember]
        [Column("width")]
        public int? Width { get; set; }

        [DataMember]
        [Column("height")]
        public int? Height { get; set; }

        [DataMember]
        [Column("net_weight")]
        public double? NetWeight { get; set; }

        [DataMember]
        [Column("gross_weight")]
        public double? GrossWeight { get; set; }

        [DataMember]
        [Column("quality_certificate")]
        public string QualityCertificate { get; set; }

        [DataMember]
        [Column("standard_number")]
        public string StandardNumber { get; set; }

        [DataMember]
        [Column("cost_price")]
        public double? CostPrice { get; set; }
    }
}


using System;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using System.Runtime.Serialization;

namespace ClassLibrary.Classes
{
    [DataContract]
    [Table("products_sales", Schema = "app")]
    public class ProductSale
    {
        [DataMember]
        [Key]
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
        [Column("id")]
        public int Id { get; set; }

        [DataMember]
        [Column("partner_id")]
        public int PartnerId { get; set; }

        [DataMember]
        [Column("production_id")]
        public int ProductionId { get; set; }

        [DataMember]
        [Column("count")]
        public double Count { get; set; }

        [DataMember]
        [Column("date")]
        public DateOnly SaleDate { get; set; }

        [DataMember]
        [Column("total_price")]
        public double? TotalPrice { get; set; }
    }
}


namespace ClassLibrary.DTOs
{
    public class PartnerDto
    {
        public int Id { get; set; }
```

```
        public string Type { get; set; }
        public string Name { get; set; }
        public int Discount { get; set; }
        public string DirectorName { get; set; }
        public string Phone { get; set; }
        public int Rating { get; set; }
    }
}


using System;

namespace ClassLibrary.DTOs
{
    public class ProductSalesDto
    {
        public int Id { get; set; }
        public string Date { get; set; }
        public double Count { get; set; }
        public string Production { get; set; }
        public double TotalPrice { get; set; }
    }
}
```

Основные этапы разработки программного обеспечения подсистемы работы с партнерами:

1. Анализ предметной области. На этом этапе изучены: деятельность компании, проблемы традиционных методов учёта, требования к системе, входные и выходные данные.

2. Проектирование. На этом этапе создали: функциональную, логическую, физическую модели.

3. Реализация. На этом этапе создали: базу данных, пользовательский интерфейс, запросы и отчёты. Выбор технологий и средств разработки зависел от специфических требований системы и доступных ресурсов.

4. Тестирование. На этом этапе проверена разработанная подсистема для работы с партнерами на предмет соответствия требованиям и выявления ошибок. Также на этапе проведено автоматическое тестирование и использованы инструменты для отладки и анализа производительности.

В рамках проекта разработаны формы ввода, просмотра, удаления, добавления и редактирования данных. Программный код форм для ввода, просмотра, удаления, добавления и редактирования данных:

Этапы разработки интерфейса:

1. Разработка интерфейса подсистемы работы с партнерами состоит из нескольких этапов. Цель — обеспечить удобство и эффективность взаимодействия пользователя с системой, сделать интерфейс интуитивно понятным и лёгким в использовании.

2. Анализ требований. На этом этапе собираются и систематизируются требования к интерфейсу со стороны потенциальных пользователей. Важно учесть потребности

конечного пользователя, чтобы упростить процессы добавления, модификации и поиска данных.

3. Проектирование. На этом этапе создаются макеты интерфейса, выбираются элементы управления, выполняется настройка их функциональности и обеспечение эргономики. После определения основных концепций проекта разрабатывается прототип интерфейса, который отражает некоторые аспекты его функционирования. В зависимости от уровня сложности прототип можно представить в виде иллюстраций интерфейса или специальных схем.

4. Реализация. На этом этапе создаются формы для удобного ввода, редактирования и просмотра информации о партнерах. Важно организовать код по модулям для обеспечения удобства поддержки и расширения функционала системы.

5. Тестирование. На этом этапе проводится тестирование разработанного интерфейса на предмет соответствия требованиям и выявления ошибок. В ходе тестирования получаются результаты: выявление ошибок, оценка удобства использования, проверка соответствия требованиям, рекомендации по улучшению.

Программный код интерфейса главного окна:

```
<Window x:Class="MasterFloor.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:MasterFloor"
        mc:Ignorable="d"
        Title="MainWindow" Height="450" Width="1200">
    <Grid>
        <Grid Margin="10">
            <Grid.RowDefinitions>
                <RowDefinition Height="50"/>
                <RowDefinition Height="*"/>
            </Grid.RowDefinitions>
            <Grid Margin="10" Grid.Row="0">
                <Menu x:Name="Menu" VerticalAlignment="Center">
                    <MenuItem Header="Файл">
                        <MenuItem Header="Сохранить Excel">
                            <MenuItem Header="Все сразу" Click="ToExcel" Com-
mandParameter="0"/>
                            <MenuItem Header="Партнеры" Click="ToExcel" Com-
mandParameter="1"/>
                            <MenuItem Header="История покупок" Click="ToExcel"
CommandParameter="2"/>
                        </MenuItem>
                    </MenuItem>
                    <MenuItem Header="Резервное копирование">
                        <MenuItem Header="Сохранить">
                            <MenuItem Header="BIN" Click="Save"/>
                            <MenuItem Header="Excel" Click="ExcelSave"/>
                        </MenuItem>
                        <MenuItem Header="Загрузить">
                            <MenuItem Header="BIN" Click="Load"/>
```

```
                            <MenuItem Header="Excel" Click="ExcelLoad"/>
                        </MenuItem>
                    </MenuItem>
                    <MenuItem Header="Продукция">
                        <MenuItem Header="Добавить" Click="AddProduction"/>
                        <MenuItem Header="Изменить" Click="RedactProduction"/>
                        <MenuItem Header="Удалить" Click="DeleteProduction"/>
                    </MenuItem>
                    <TextBox x:Name="Source_TB" Width="100" HorizontalAlign-
ment="Left" VerticalAlignment="Center" TextChanged="Source_TB_TextChanged"/>
                </Menu>
            </Grid>
            <Grid Margin="10" Grid.Row="1">
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="*" MinWidth="300"/>
                    <ColumnDefinition Width="2*" MaxWidth="800"/>
                </Grid.ColumnDefinitions>
                <Grid Margin="10, 10, 10, 10" Grid.Column="0">
                    <ListBox x:Name="Partners_LB" Padding="10" Selection-
Changed="Partners_LB_Selected">
                        <ListBox.ItemTemplate>
                            <DataTemplate>
                                <Border Padding="5, 5, 35, 5" Height="110" Ver-
ticalAlignment="Center"
                                        BorderBrush="Black" BorderThick-
ness="0.5" HorizontalAlignment="Stretch"
                                        Width="{Binding ActualWidth, Rela-
tiveSource={RelativeSource AncestorType=ListBox}}">
                                    <StackPanel Margin="5" HorizontalAlign-
ment="Stretch">
                                        <Grid Margin="5" HorizontalAlign-
ment="Stretch">
                                            <StackPanel HorizontalAlign-
ment="Left" Orientation="Horizontal">
                                                <TextBlock FontSize="14" Pad-
ding="5, 5, 0, 5" Text="{Binding Path=Type}" HorizontalAlignment="Left" Verti-
calAlignment="Top"/>
                                                <Label Content="|"/>
                                                <TextBlock FontSize="14" Pad-
ding="0, 5, 5, 5" Text="{Binding Path=Name}" HorizontalAlignment="Left" Verti-
calAlignment="Top"/>
                                            </StackPanel>
                                            <TextBlock FontSize="12" Pad-
ding="5" Text="{Binding Path=Discount, StringFormat={}{0} %}" HorizontalAlign-
ment="Right" VerticalAlignment="Top"/>
                                        </Grid>
                                        <StackPanel Orientation="Vertical" Mar-
gin="5" HorizontalAlignment="Stretch">
                                            <TextBlock FontSize="12"
Text="{Binding Path=DirectorName, StringFormat={}Директор {0}}" Horizonta-
lAlignment="Left" />
                                            <TextBlock FontSize="12"
Text="{Binding Path=Phone}" HorizontalAlignment="Left" />
                                            <TextBlock FontSize="12"
Text="{Binding Path=Rating, StringFormat={}Рейтинг: {0}}" HorizontalAlign-
ment="Left" />
                                        </StackPanel>
                                    </StackPanel>
                                </Border>
                            </DataTemplate>
                        </ListBox.ItemTemplate>
                        <ListBox.ContextMenu>
```

```
                            <ContextMenu>
                                <MenuItem Header="Добавить"
Click="AddPartner"/>
                                <MenuItem Header="Редактировать"
Click="RedactPartner"/>
                                <MenuItem Header="Удалить"
Click="DeletePartner"/>
                                <MenuItem Header="Добавить заказ"
Click="AddProductSaleSelected"/>
                            </ContextMenu>
                        </ListBox.ContextMenu>
                    </ListBox>
                </Grid>
                <Grid Margin="10" Grid.Column="1">

                    <DataGrid x:Name="ProductSales_Table" CanUserSortCol-
umns="True" Margin="10, 10, 10, 10" AutoGenerateColumns="False" CanUserAd-
dRows="False" CanUserDeleteRows="False" CanUserReorderColumns="False" IsReadOn-
ly="True" RowBackground="Azure" AlternatingRowBackground="White">
                        <DataGrid.Columns>
                            <DataGridTextColumn Width="*" Header="Дата" Bind-
ing="{Binding Path=Date}"/>
                            <DataGridTextColumn Width="*" Header="Количество"
Binding="{Binding Path=Count}"/>
                            <DataGridTextColumn Width="*" Header="Продукция"
Binding="{Binding Path=Production}"/>
                            <DataGridTextColumn Width="*" Header="Итоговая сум-
ма" Binding="{Binding Path=TotalPrice, StringFormat={}{0} ₽}"/>
                        </DataGrid.Columns>
                        <DataGrid.ContextMenu>
                            <ContextMenu>
                                <MenuItem Header="Добавить"
Click="AddProductSale"/>
                                <MenuItem Header="Редактировать"
Click="RedactProductSale"/>
                                <MenuItem Header="Удалить"
Click="DeleteProductSale"/>
                            </ContextMenu>
                        </DataGrid.ContextMenu>
                    </DataGrid>
                </Grid>
            </Grid>
        </Grid>
    </Grid>
</Window>
```

Интерфейс главного окна приложения представлен на рисунке 2.

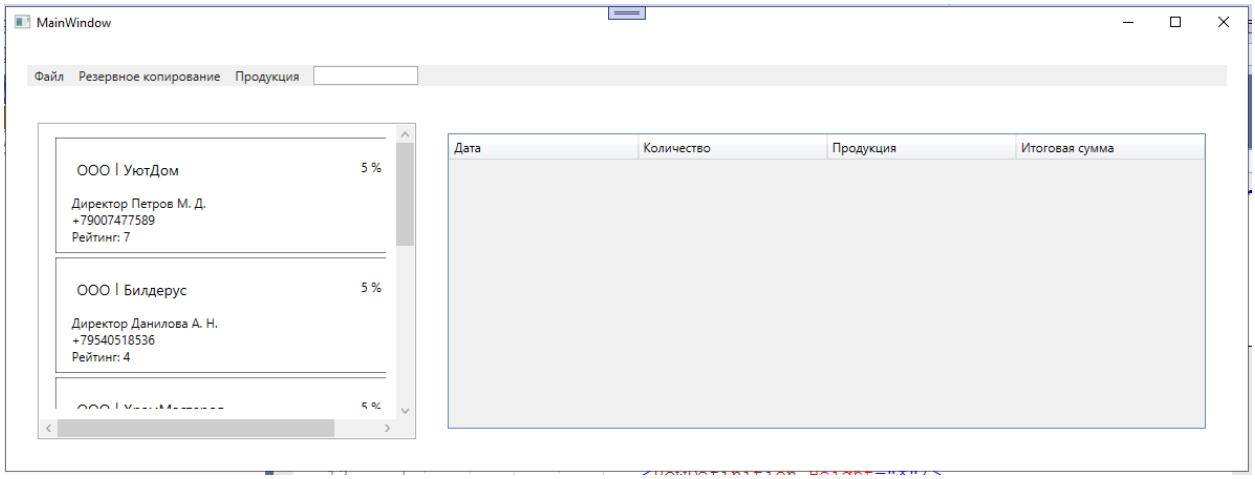

Рисунок 2 – Скриншот интерфейса главного окна приложения

Программный код интерфейса формы изменения данных о партнерах:

```
<Window x:Class="MasterFloor.DataForms.PartnerRedactWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:MasterFloor.DataForms"
        mc:Ignorable="d"
        Title="PartnerRedactWindow" Height="500" Width="700">
    <Grid>
        <Grid Margin="10, 10, 10, 10">
            <Grid Margin="10, 10, 10, 10">
                <Grid Margin="10" Grid.Row="0">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="*"/>
                    </Grid.ColumnDefinitions>
                    <Grid.RowDefinitions>
                        <RowDefinition Height="*"/>
                        <RowDefinition Height="*"/>
                        <RowDefinition Height="*"/>
                        <RowDefinition Height="*"/>
                        <RowDefinition Height="*"/>
                    </Grid.RowDefinitions>
                    <Grid Margin="10, 10, 10, 10"  Grid.Row="0"
Grid.Column="0">
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="*"/>
                            <ColumnDefinition Width="2*"/>
                        </Grid.ColumnDefinitions>
                        <TextBlock TextWrapping="Wrap" Margin="10" Text="Тип:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                        <ComboBox x:Name="Partner_Type_CB" Grid.Column="1">
                            <ComboBoxItem Content="ИП"/>
                            <ComboBoxItem Content="КФЛ"/>
                            <ComboBoxItem Content="ХТ"/>
                            <ComboBoxItem Content="ХО"/>
                            <ComboBoxItem Content="ООО"/>
                            <ComboBoxItem Content="АО/НАО"/>
                            <ComboBoxItem Content="ПАО"/>
                            <ComboBoxItem Content="ХП"/>
                            <ComboBoxItem Content="УП"/>
```

```
                        </ComboBox>
                    </Grid>
                    <Grid Margin="10, 10, 10, 10"  Grid.Row="0"
Grid.Column="1">
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="*"/>
                            <ColumnDefinition Width="2*"/>
                        </Grid.ColumnDefinitions>
                        <TextBlock TextWrapping="Wrap" Margin="10"
Text="Наименование:" HorizontalAlignment="Right" VerticalAlignment="Center"
Grid.Column="0"/>
                        <TextBox x:Name="Partner_Name_TB" Grid.Column="1"/>
                    </Grid>
                    <Grid Margin="10, 10, 10, 10"  Grid.Row="1"
Grid.Column="0">
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="*"/>
                            <ColumnDefinition Width="2*"/>
                        </Grid.ColumnDefinitions>
                        <TextBlock TextWrapping="Wrap" Margin="10" Text="Юр.
адрес:" HorizontalAlignment="Right" VerticalAlignment="Center"
Grid.Column="0"/>
                        <TextBox x:Name="Partner_Legal_Address_TB"
Grid.Column="1"/>
                    </Grid>
                    <Grid Margin="10, 10, 10, 10"  Grid.Row="1"
Grid.Column="1">
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="*"/>
                            <ColumnDefinition Width="2*"/>
                        </Grid.ColumnDefinitions>
                        <TextBlock TextWrapping="Wrap" Margin="10" Text="ИНН:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                        <TextBox x:Name="Partner_INN_TB" Grid.Column="1"/>
                    </Grid>
                    <Grid Margin="10, 10, 10, 10"  Grid.Row="2"
Grid.Column="0">
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="*"/>
                            <ColumnDefinition Width="2*"/>
                        </Grid.ColumnDefinitions>
                        <TextBlock TextWrapping="Wrap" Margin="10"
Text="Директор:" HorizontalAlignment="Right" VerticalAlignment="Center"
Grid.Column="0"/>
                        <TextBox x:Name="Partner_Director_Name_TB"
Text="(Фамилия + инициалы)" Grid.Column="1"/>
                    </Grid>
                    <Grid Margin="10, 10, 10, 10"  Grid.Row="2"
Grid.Column="1">
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="*"/>
                            <ColumnDefinition Width="2*"/>
                        </Grid.ColumnDefinitions>
                        <TextBlock TextWrapping="Wrap" Margin="10"
Text="Телефон:" HorizontalAlignment="Right" VerticalAlignment="Center"
Grid.Column="0"/>
                        <TextBox x:Name="Partner_Phone_TB" Grid.Column="1"/>
                    </Grid>
                    <Grid Margin="10, 10, 10, 10"  Grid.Row="3"
Grid.Column="0">
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="*"/>
```

```
                            <ColumnDefinition Width="2*"/>
                        </Grid.ColumnDefinitions>
                        <TextBlock TextWrapping="Wrap" Margin="10" Text="E-
mail:" HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                        <TextBox x:Name="Partner_Email_TB" Grid.Column="1"/>
                    </Grid>
                    <Grid Margin="10, 10, 10, 10"  Grid.Row="3"
Grid.Column="1">
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="*"/>
                            <ColumnDefinition Width="2*"/>
                        </Grid.ColumnDefinitions>
                        <TextBlock TextWrapping="Wrap" Margin="10"
Text="Логотип:" HorizontalAlignment="Right" VerticalAlignment="Center"
Grid.Column="0"/>
                        <Button x:Name="Partner_Logo_B"
Click="Partner_Logo_Choosing" Margin="15, 10" Content="Выберите файл..."
Grid.Column="1"/>
                    </Grid>
                    <Grid Margin="10, 10, 10, 10"  Grid.Row="4"
Grid.Column="0">
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="*"/>
                            <ColumnDefinition Width="2*"/>
                        </Grid.ColumnDefinitions>
                        <TextBlock TextWrapping="Wrap" Margin="10"
Text="Рейтинг:" HorizontalAlignment="Right" VerticalAlignment="Center"
Grid.Column="0"/>
                        <TextBox x:Name="Partner_Rating_TB" Grid.Column="1"/>
                    </Grid>
                    <Grid Margin="10, 10, 10, 10" Grid.Row="4" Grid.Column="1">
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="*"/>
                            <ColumnDefinition Width="*"/>
                        </Grid.ColumnDefinitions>
                        <Button x:Name="accept_btn" Content="Добавить" Horizon-
talAlignment="Center" VerticalAlignment="Center" Click="accept_btn_Click"
Grid.Column="0"/>
                        <Button x:Name="reject_btn" Content="Отмена" Horizonta-
lAlignment="Center" VerticalAlignment="Center" Click="reject_btn_Click"
Grid.Column="1"/>
                    </Grid>
                </Grid>
            </Grid>
        </Grid>
    </Grid>
</Window>
```

Форма изменения данных о партнерах представлена на рисунке 3.

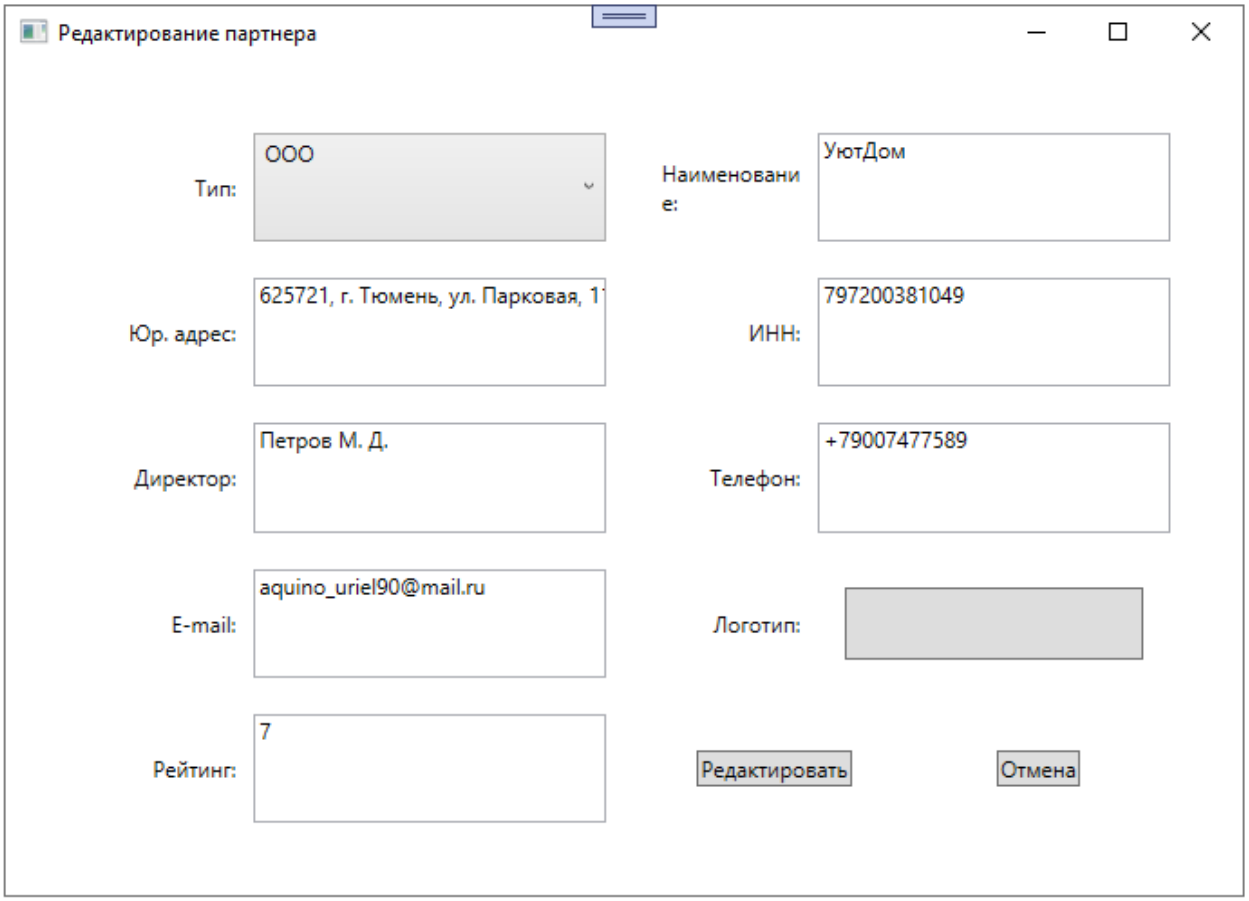

Рисунок 3 - Скриншот формы изменения данных о партнерах

Программный код интерфейса формы изменения данных о продукции:

```
<Window x:Class="MasterFloor.DataForms.ProductionRedactWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:MasterFloor.DataForms"
        mc:Ignorable="d"
        Title="ProductionRedactWindow" Height="450" Width="1200">
    <Grid>
        <Grid Margin="10, 10, 10, 10">
            <Grid Margin="10, 10, 10, 10">
                <Grid.RowDefinitions>
                    <RowDefinition Height="*"/>
                    <RowDefinition Height="*"/>
                    <RowDefinition Height="*"/>
                    <RowDefinition Height="*"/>
                    <RowDefinition Height="*"/>
                </Grid.RowDefinitions>
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="*"/>
                    <ColumnDefinition Width="*"/>
                    <ColumnDefinition Width="*"/>
                </Grid.ColumnDefinitions>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="0" Grid.Column="0">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
```

```
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Артикль:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <TextBox x:Name="Production_Article_TB" Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="0" Grid.Column="1">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10"
Text="Наименование:" HorizontalAlignment="Right" VerticalAlignment="Center"
Grid.Column="0"/>
                    <TextBox x:Name="Production_Name_TB" Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="0" Grid.Column="2">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Описание:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <TextBox x:Name="Production_Description_TB"
Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="1" Grid.Column="0">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Фото:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <Button x:Name="Production_Photo_B"
Click="Production_Logo_Choosing" Margin="15, 10" Content="Выберите файл..."
Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="1" Grid.Column="1">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Масса
нетто:" HorizontalAlignment="Right" VerticalAlignment="Center"
Grid.Column="0"/>
                    <TextBox x:Name="Production_Net_Weight_TB"
Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="1" Grid.Column="2">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Масса
брутто:" HorizontalAlignment="Right" VerticalAlignment="Center"
Grid.Column="0"/>
                    <TextBox x:Name="Production_Gross_Weight_TB"
Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="2" Grid.Column="0">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
```

```
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Длинна:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <TextBox x:Name="Production_Length_TB" Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="2" Grid.Column="1">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Ширина:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <TextBox x:Name="Production_Width_TB" Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="2" Grid.Column="2">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Высота:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <TextBox x:Name="Production_Height_TB" Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="3" Grid.Column="0">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10"
Text="Минимальная цена:" HorizontalAlignment="Right" VerticalAlignment="Center"
Grid.Column="0"/>
                    <TextBox x:Name="Production_Min_Partner_Price_TB"
Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="3" Grid.Column="1">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Сертификат
качества:" HorizontalAlignment="Right" VerticalAlignment="Center"
Grid.Column="0"/>
                    <TextBox x:Name="Production_Quality_Certificate_TB"
Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="3" Grid.Column="2">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Номер ГОС-
Та:" HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <TextBox x:Name="Production_Standard_Number_TB"
Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="4" Grid.Column="0">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Цена:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
```

```
                    <TextBox x:Name="Production_Cost_Price_TB"
Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="4" Grid.Column="1">
                    <Button x:Name="accept_btn" Content="Добавить" Horizonta-
lAlignment="Center" VerticalAlignment="Center" Click="accept_btn_Click"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="4" Grid.Column="2">
                    <Button x:Name="reject_btn" Content="Отмена" Horizonta-
lAlignment="Center" VerticalAlignment="Center" Click="reject_btn_Click"/>
                </Grid>
            </Grid>
        </Grid>
    </Grid>
</Window>
```

Форма изменения данных о продукции представлена на рисунке 4.

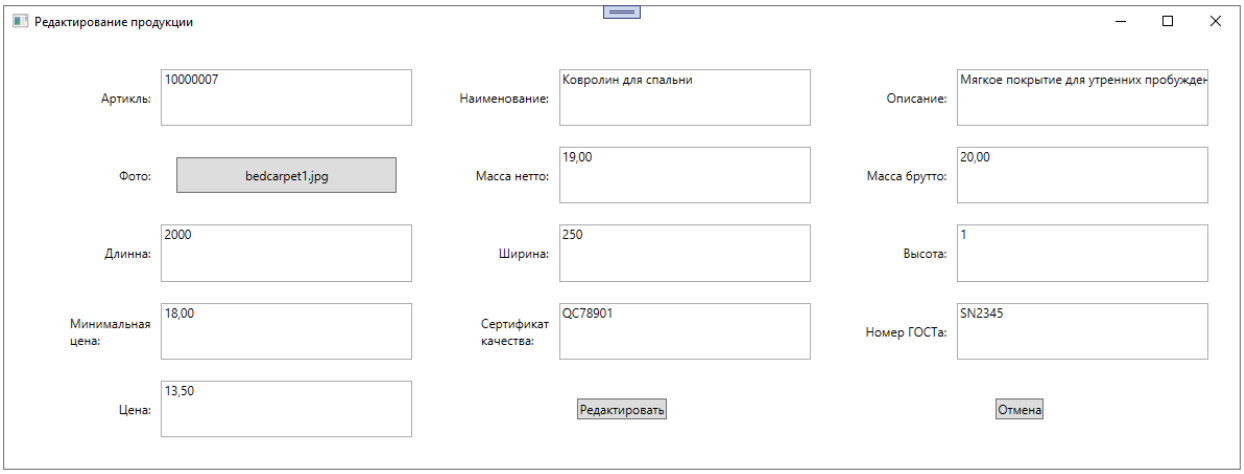

Рисунок 4 - Скриншот формы изменения данных о продукции

Программный код интерфейса формы изменения данных о закупках:

```
<Window x:Class="MasterFloor.DataForms.ProductSaleRedactWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:MasterFloor.DataForms"
        mc:Ignorable="d"
        Title="ProductSaleRedactWindow" Height="450" Width="400">
    <Grid>
        <Grid Margin="10, 10, 10, 10">
            <Grid Margin="10, 10, 10, 10">
                <Grid.RowDefinitions>
                    <RowDefinition Height="*"/>
                    <RowDefinition Height="*"/>
                    <RowDefinition Height="*"/>
                    <RowDefinition Height="*"/>
                    <RowDefinition Height="*"/>
                </Grid.RowDefinitions>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="0">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
```

```
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Партнер:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <ComboBox x:Name="ProductSale_Partner_CB" Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="1">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10"
Text="Продукция:" HorizontalAlignment="Right" VerticalAlignment="Center"
Grid.Column="0"/>
                    <ComboBox x:Name="ProductSale_Production_CB"
Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="2">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10"
Text="Количество:" HorizontalAlignment="Right" VerticalAlignment="Center"
Grid.Column="0"/>
                    <TextBox x:Name="ProductSale_Count_TB" Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="3">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Дата:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <DatePicker x:Name="ProductSale_Date_DP" Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10" Grid.Row="4">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="*"/>
                    </Grid.ColumnDefinitions>
                    <Button x:Name="accept_btn" Content="Добавить" Horizonta-
lAlignment="Center" VerticalAlignment="Center" Click="accept_btn_Click"
Grid.Column="0"/>
                    <Button x:Name="reject_btn" Content="Отмена" Horizonta-
lAlignment="Center" VerticalAlignment="Center" Click="reject_btn_Click"
Grid.Column="1"/>
                </Grid>
            </Grid>
        </Grid>
    </Grid>
</Window>
```

Форма изменения данных о закупках представлена на рисунке 5.

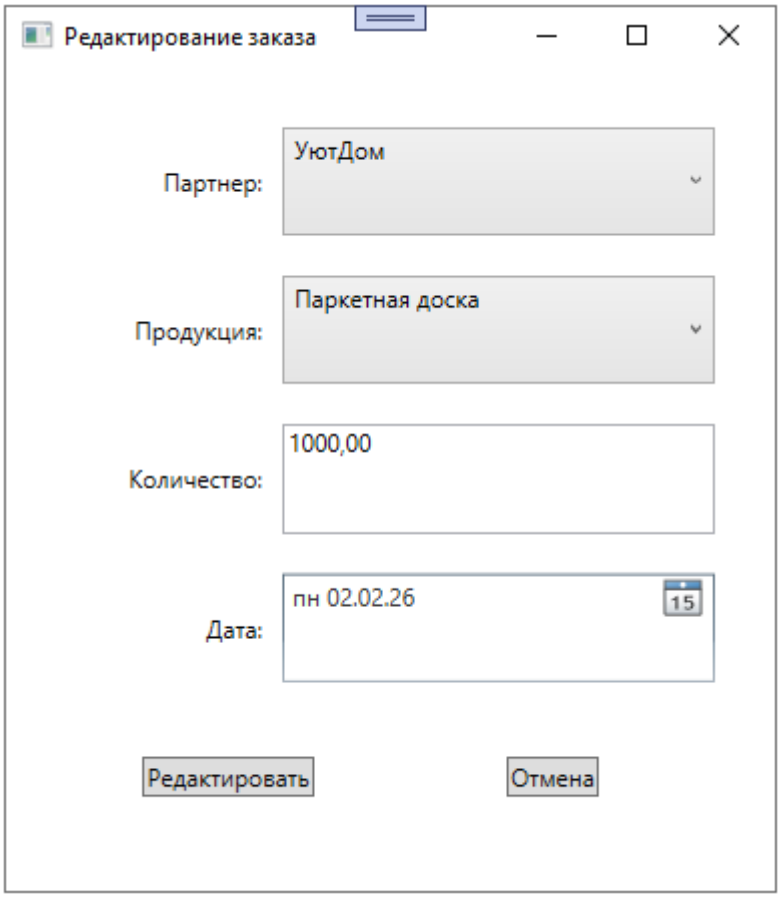

Рисунок 5 - Скриншот формы изменения данных о закупках

3.1.3. Использование методов для получения кода с заданной функциональностью и степенью качества

При разработке приложения для работы с партнерами использовалось несколько методов получения кода, обеспечивающие соответствие заданной функциональности и необходимому уровню качества, такие как:

1. Анализ требований и проектирование:

1.1. Сбор требований от заказчика для четкого понимания функциональных задач.

1.2. Создание технического задания, определяющего функциональные и нефункциональные требования.

1.3. Разработка архитектурных решений и проектной документации, формирующих основу для последующей реализации.

1.4. Автоматизированное тестирование: разработка и внедрение юнит unit-тестов.

2. Использование систем контроля версий: ведение кода в системе контроля версий (Git) позволяет отслеживать историю изменений, управлять ветками и обеспечивать отмену ошибок.

3. Документирование кода и написание комментариев для повышения понимания и поддержки.

Для получения кода с необходимой функциональностью и высоким уровнем качества при разработке приложения использовался комплексный подход, включавший анализ требований, применение стандартных шаблонов, автоматические тесты и системы контроля версий. Такой системный подход позволил обеспечить не только реализацию требуемого функционала, но и поддерживать качество, надежность и расширяемость создаваемого программного продукта.

3.1.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

Инспекция программного обеспечения на предмет соответствия стандартам кодирования — это важный этап контроля качества и поддерживаемости проекта. Она позволяет выявить несоответствия, ошибки и потенциальные уязвимости до этапа тестирования и внедрения. В зависимости от цели, методов и степени автоматизации, существуют различные виды инспекций.

Виды инспекций программного обеспечения на предмет соответствия стандартам кодирования:

1. Ручная инспекция (Code Review)

Описание: Команда разработчиков или специалистов по качеству кода вручную просматривает код на соответствие установленным стандартам.

Цели: обнаружить нарушения стиля, логические ошибки, потенциальные уязвимости, недочеты в архитектуре.

Преимущества: возможность тщательного анализа, выявление сложных ошибок, обмен знаниями внутри команды.

Недостатки: затратность по времени, субъективность оценки, качество зависит от компетентности рецензентов.

2. Инспекция с помощью автоматизированных инструментов (Static Code Analysis)

Описание: использование специальных программных средств, автоматически сканирующих код на соответствие стандартам, правилам и лучшим практикам.

Примеры инструментов: SonarQube, ESLint, Pylint, Checkstyle, Coverity, PMD.

Цели: обнаружить стилистические нарушения, дублирование кода, потенциально опасные участки, ошибки безопасности.

Преимущества: быстрое выполнение, систематичность, возможность интеграции в CI/CD процессы.

Недостатки: возможны ложные срабатывания, не заменяют полноценную ручную инспекцию.

3. Формальные инспекции (Formal Inspection)

Описание: структурированный процесс проверки, включающий подготовку, проведение инспекции, фиксацию нарушений и последующее исправление.

Цели: единовременное, точное обнаружение отклонений от стандартов, анализ возможных последствий.

Преимущества: высокая точность, систематический подход, возможность обучающего обмена опытом.

Недостатки: требует много ресурсов, времени и организационных усилий.

4. Код-ревью через парное программирование (Pair Programming Review)

Описание: два разработчика совместно пишут и просматривают код в реальном времени.

Цели: моментальное исправление ошибок, соблюдение стандартов, обмен знаниями.

Преимущества: повышенная внимательность, быстрая обратная связь.

Недостатки: требует участия двух специалистов одновременно, возможна субъективность.

5. Статический анализ кода (Static Analysis) с автоматическими правками

Описание: автоматические системы не только ищут нарушения, но и предлагают или автоматически исправляют некоторые из них.

Цели: ускорить процесс исправления, обеспечить постоянное соблюдение стандартов.

Преимущества: высокая скорость и систематизм.

Недостатки: возможные ошибки исправлений, требуется настройка.

Инспекции программного обеспечения на соответствие стандартам кодирования — это та часть процесса обеспечения качества программного продукта, которая включает в себя как ручные, так и автоматические методы. Каждый вид имеет свои преимущества и ограничения, и наиболее эффективным является их комбинирование: автоматизированный анализ для быстрого поиска очевидных нарушений и ручной просмотр для глубокого анализа и исправления сложных ошибок. Такой комплексный подход гарантирует, что код

будет не только функциональным, но и легко поддерживаемым, безопасным и соответствующим принятым стандартам

Актуальные стандарты кодирования:

– Один из актуальных стандартов кодирования — Unicode. Это универсальный стандарт, который охватывает практически все письменные языки планеты. Он присвоил уникальный код каждому символу, включая буквы, цифры, знаки препинания, иероглифы, эмодзи и многое другое.

–Также используются ASCII-совместимые кодировки. Они включают в себя все символы ASCII и добавляют к ним другие символы, например, символы кириллицы, греческого алфавита, японских иероглифов.

–EBCDIC - кодировка, разработанная IBM. Менее популярна, чем ASCII-совместимые кодировки, но всё ещё используется в некоторых системах IBM.

–SOLID это набор принципов объектно-ориентированного программирования, которые помогают писать более качественный и поддерживаемый код.

Кроме того, в России и мире применяются национальные стандарты, которые регламентируют использование кодировок в определённых областях. Некоторые из них:

- ГОСТ 27463-87 — «Системы обработки информации». Стандарт устанавливает общие принципы и требования к системам обработки информации, включая вопросы кодирования данных.

- ГОСТ 27465-87 — «Системы обработки информации». Стандарт устанавливает требования к форматам представления данных, включая кодировку символов.

- ГОСТ Р 51001-96, ГОСТ Р 51002-96 и ГОСТ Р 51003-96 - «Автоматическая идентификация». Стандарты описывают системы автоматической идентификации, в которых кодирование играет важную роль.

Правила оформления кода в программировании включают рекомендации по структурированию, именованию элементов и использованию комментариев. Эти правила помогают улучшить читаемость и поддерживаемость кода, а также сократить время на отладку и тестирование:

1. Именование:

➢ Использовать описательные имена для переменных, функций и констант.

➢ Избегать зарезервированных ключевых слов в именах — это может привести к ошибкам компиляции. Перед именованием элементов рекомендуется проверить документацию на список зарезервированных ключевых слов.

2. Структура:

➢ Использовать фигурные скобки для объединения части кода в единый блок.

➢ Располагать один оператор на строке — это облегчает чтение кода сверху вниз. Если аргументы функции не помещаются на одной строке, их следует разбивать на несколько строк.

➢ В сложных выражениях размещать каждое условие на отдельной строке.

3. Комментарии:

➢ Использовать комментарии для пояснения кода, но не для всего подряд. Рекомендуется оставлять комментарии только для значимых элементов: функций, модулей, констант, глобальных переменных.

➢ Место написания — комментарии пишутся справа от строки, к которой они относятся, если комментарий короткий, или сверху над кодом, если комментарий многострочный.

➢ Размер комментария — не рекомендуется составлять длинные комментарии, многострочный комментарий должен быть лаконичным.

4. Ошибки

➢ Проверять возвращаемые значения функций — многие системные функции возвращают специальные значения, которые указывают на успех или неудачу операции.

➢ Использовать отладчик для пошагового выполнения программы и анализа значений переменных.

➢ Писать тесты для кода, чтобы убедиться, что он работает правильно.

Требования к структуре кода на включают правила именования, форматирования, построения логики и обработки ошибок. Эти требования помогают писать читаемый и поддерживаемый код, а также минимизировать ошибки:

1. Именование. Использовать описательные имена для переменных, функций и констант:

• Функции — имена, которые чётко описывают, что делает функция (шаблон «глагол-существительное»).

• Переменные — имена, отражающие, что именно хранится, а не какое действие выполняется. Следует избегать односимвольных имён переменных, за исключением временных «одноразовых».

• Константы — пишутся заглавными буквами, разделёнными подчёркиванием.

2. Форматирование

• Использовать последовательное отступы (обычно 2 или 4 пробела) для чёткого показа структуры кода.

• Добавлять пробелы вокруг операторов и после запятых для улучшения читаемости.

- Ограничивать длину строк.
- Группировать связанный код вместе.

3. Логика

- Функции — короткие, сосредоточенные на одной задаче.
- Условные операторы — позволяют выполнять код в зависимости от условий.
- Циклы — позволяют выполнять код многократно, используются для повторения одних и тех же действий.
- Переходные операторы — управляют потоком выполнения внутри циклов и условий.

4. Ошибки

- Проверять возвращаемые значения функций — многие системные функции возвращают специальные значения, которые указывают на успех или неудачу операции.
- Использовать глобальную переменную errno — она устанавливается системными функциями для указания на тип возникшей ошибки. После неудачного вызова функции можно обратиться к errno с помощью функции perror или strerror, чтобы получить более подробную информацию о причинах неправильной работы кода.
- Проверять граничные условия и случаи, которые могут вызвать ошибки.

Требования к организации файлов в проекте включают правила структуры каталогов, именования файлов, версионирования и меры безопасности. Эти требования помогают улучшить читаемость кода, облегчить совместную работу и снизить риски:

1. Структура каталогов

- Использовать осмысленные имена папок для разных частей проекта.
- Группировать связанные файлы вместе, чтобы улучшить организацию кода.
- Использовать подпапки для сложных модулей или слоёв в больших проектах.
- Избегать размещения сгенерированных или промежуточных файлов непосредственно в каталоге исходного кода — создавать отдельный каталог для этих файлов. Это помогает дифференцировать между фактическими исходными файлами и сгенерированными.

2. Имена файлов

- Использовать единые имена для файлов, которые отражают их назначение.
- Соблюдать постоянство в использовании разделителей (подчёркивания или тире) внутри одного проекта. Не должно быть так, что часть файлов используют подчёркивания, а часть — тире.

- Не использовать пробелы в названии файла — некоторые файловые менеджеры и библиотеки не поддерживают пробелы.

3. Версионирование

- Использовать семантическое версионирование (SemVer) — соглашение об именовании, которое обозначает ключевые вехи. Базовый вариант — трёхкомпонентный формат MAJOR.MINOR.PATCH.
- Централизовать информацию о версии в заголовочном файле, чтобы обновлять её в одном месте и получать доступ к ней из нескольких исходных файлов.
- Настроить системы сборки (например, CMake, Makefile) так, чтобы включать информацию о версии во время процесса сборки.

4. Безопасность

- Избегать размещения зашифрованных учётных данных в исходном коде — вместо этого использовать заполнители в источнике и позволить системе развёртывания заменить их секретами из утверждённых хранилищ.
- Отслеживать внешние зависимости и устранять уязвимости безопасности в этих компонентах, обновляя их до более безопасных версий, когда они подвергаются этим уязвимостям.

Для инспекции программного обеспечения на предмет соответствия стандартам кодирования используются как бесплатные, так и платные программы. Цель инспекции — обнаружить и исправить ошибки, которые были пропущены при разработке.

Наиболее распространенные программы, используемые для осуществления инспекции программного обеспечения на предмет соответствия стандартам кодирования:

- SonarLint — расширение с открытым исходным кодом для интеграции в популярные среды разработки (Visual Studio Code, IntelliJ IDEA, Eclipse). Анализирует код в реальном времени, идентифицирует проблемы: от стилистических погрешностей и дублирующихся фрагментов до отклонений от установленных правил кодирования. Информирует разработчика о найденных проблемах, часто предоставляет рекомендации по их устранению.
- Java Coding Standard Checker (JCSC) — настраиваемый инструмент проверки исходного кода Java. Проверяет соответствие определяемым стандартам кодирования, таким как соглашения об именовании и структура кода. Обнаруживает признаки плохого кодирования и потенциальные ошибки.
- ESLint — инструмент для анализа и улучшения кода, написанного на JavaScript и ECMAScript. Помогает выявлять синтаксические ошибки, проблемы в шаблонах

проектирования и отклонения от стиля кодирования. Позволяет настраивать правила проверки, использовать готовые плагины и создавать собственные правила.

• Codacy — сервис для автоматического анализа и мониторинга качества исходного кода. Подходит для проверки кода на Java, Python, JavaScript, PHP и более 30 других языках. С помощью искусственного интеллекта может автоматически исправлять код.

• Code Climate — сервис для анализа кода и мониторинга качества ПО. Производит статический и автоматизированный анализ кода на уязвимости, нарушения стиля и другие проблемы без участия разработчика, а затем предоставляет выводы в виде детализированных отчётов о состоянии кода с предложениями по улучшению.

• Polyspace Bug Finder инструмент, позволяющий проверять исходный код на соотве тствие стандарту MISRA-
C и другим стандартам, а также добавлять собственные правила. Измеряет метрики исход ного кода, влияющие на читаемость, тестируемость и безопасность.

• HP Fortify Static Code Analyz

Написанный код был проверен на соответствие установленным корпоративным стандартам университета. В результате анализа установлено, что структура, стиль оформления, а также используемые подходы полностью соответствуют утвержденным рекомендациям и нормативам.

Код выполнен в соответствии с принятыми стандартами форматирования: соблюдены нормы отступов, правильное расположение скобок, единообразное именование переменных и методов, а также соблюдены требования по комментированию и документации. Стиль написания кода способствует высокой читаемости и удобству поддержки, что соответствует внутренним стандартам качества кода, принятым в университете.

3.1.5. Выполнение отладки программного модуля с использованием специализированных программных средств

Отладка программного обеспечения — это процесс поиска и исправления ошибок в программном коде. Цель отладки - найти и устранить проблему, которая мешает работе программы или приводит к неправильным результатам.

Процесс отладки включает несколько этапов:

1. Обнаружение ошибки. Определение симптомов, указывающих на наличие ошибки в программе.

2. Воспроизведение ошибки. Создание условий, при которых ошибка проявляется снова, чтобы лучше понять её природу.

3. Диагностика проблемы. Использование различных инструментов и методов для анализа кода и поиска причины ошибки.

4. Исправление ошибки. Внесение изменений в код для устранения ошибки.

5. Тестирование. Проверка исправления и тестирование программы, чтобы убедиться, что ошибка устранена и не появились новые проблемы.

Visual Studio поддерживает отладку программного модуля на C# с помощью отладчика. Отладчик позволяет запускать приложение с подключённым к нему отладчиком, отслеживать выполнение кода и проверять переменные.

В процессе проекта выполнена отладка программного обеспечения, ошибки в программном коде исправлены (рисунок 6).

```
552
553                 Partner partner = new()
554                 {
555                     Id = Convert.ToInt32(idCell),
556                     Type = Convert.ToString(partnerSheet.Cells[$"C{
557                     Name = Convert.ToString(partnerSheet.Cells[$"D{
558                     LegalAddress = Convert.ToString(partnerSheet.Ce
559                     Inn = Convert.ToString(partnerSheet.Cells[$"F{r
560                     DirectorName = Convert.ToString(partnerSheet.Ce
561                     Phone = Convert.ToString(partnerSheet.Cells[$"H
562                     Email = Convert.ToString(partnerSheet.Cells[$"I
563                     Logo = Convert.ToString(partnerSheet.Cells[$"J{
564                     Rating = Convert.ToInt32(partnerSheet.Cells[$"K
565                 };
```

Рисунок 6 – Результат отладки модуля

3.1.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения

Процесс проведения тестирования программного обеспечения включает следующие этапы:

- Проработка требований к продукту. Изучаются все аспекты продукта для определения ключевых областей, которые нужно протестировать.

- Анализ требований. Проводится углублённое изучение требований для выявления потенциальных рисков и несоответствий.

- Разработка стратегии и плана тестирования. Создаётся детализированный план, включающий подходы, методы, распределение ресурсов и график выполнения.

- Создание тестовой документации. Разрабатываются тест-кейсы, чек-листы и другая документация, которая станет основой для тестирования.

- Проведение тестирования. Реализация запланированных тестов на основе подготовленной документации. На этом этапе проводится проверка функционала, производительности и других характеристик продукта.

- Эксплуатация и поддержка. На финальном этапе проводится тестирование продукта в реальной среде. Это включает сбор обратной связи от пользователей и мониторинг работы системы в условиях, приближённых к эксплуатации.

В ходе выполнения проекта были реализованы unit-тесты (рисунок 7).

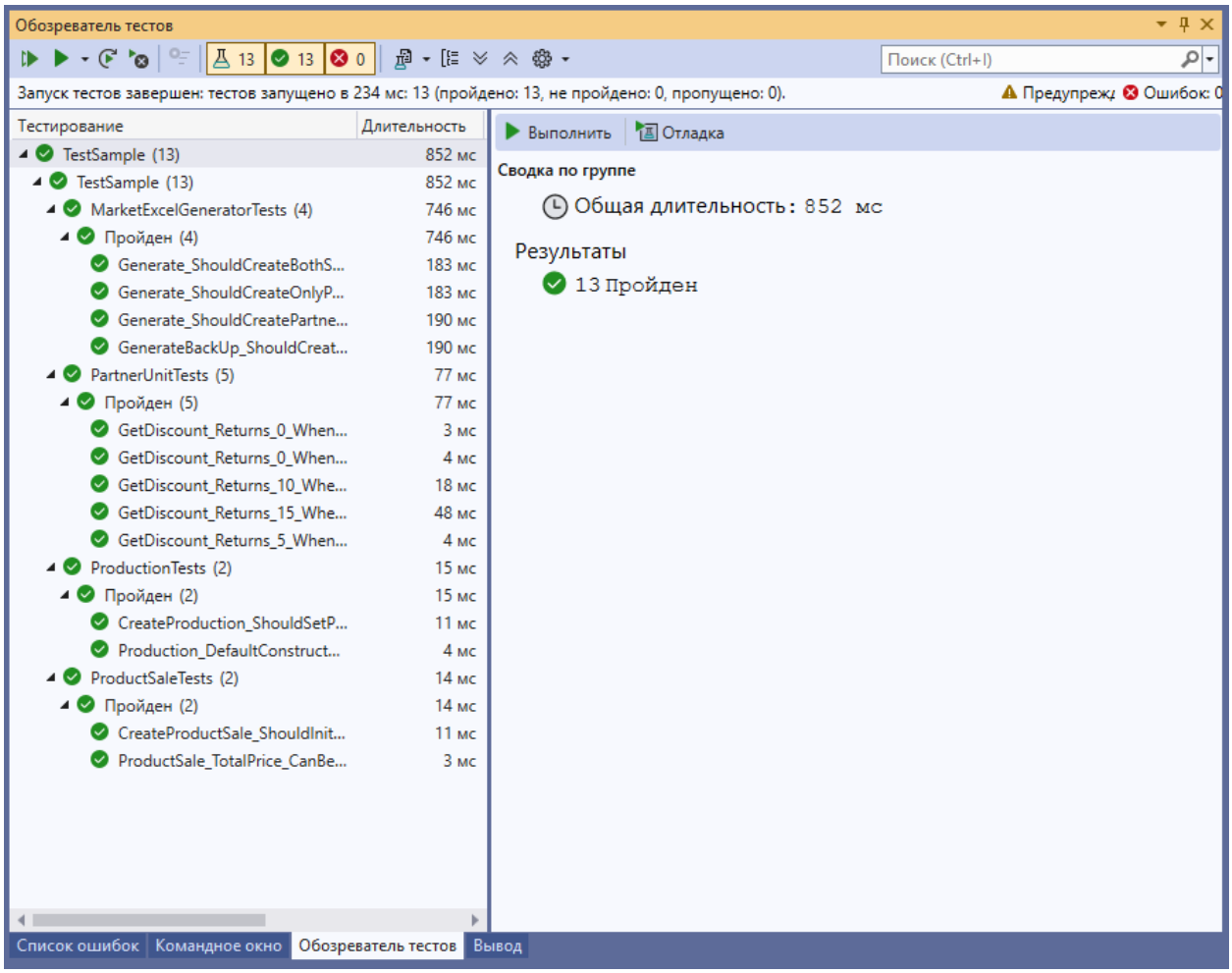

Рисунок 7 – Результат тестирования

3.1.7. Интегрирование программных модулей

Выполним интеграцию модуля истории реализации продукции партнерам. Процесс интеграции данного модуля в приложение начинается с написания кода для интерфейса. Далее проектируем показательный класс для корректного и логичного отображения данных. Внутри главного окна записываем метод для отображения истории реализации продукции, которая будет динамически обновляться. Затем пишем функционал для манипуляций историей реализации продукции (стандартные CRUD-

методы) и добавляем необходимые функции в меню.

Код интерфейса модуля истории реализации продукции партнерам:

```
                <Grid Margin="10" Grid.Column="1">
                    <DataGrid x:Name="ProductSales_Table" CanUserSortCol-
umns="True" Margin="10, 10, 10, 10" AutoGenerateColumns="False" CanUserAd-
dRows="False" CanUserDeleteRows="False" CanUserReorderColumns="False" IsReadOn-
ly="True" RowBackground="Azure" AlternatingRowBackground="White">
                        <DataGrid.Columns>
                            <DataGridTextColumn Width="*" Header="Дата" Bind-
ing="{Binding Path=Date}"/>
                            <DataGridTextColumn Width="*" Header="Количество"
Binding="{Binding Path=Count}"/>
                            <DataGridTextColumn Width="*" Header="Продукция"
Binding="{Binding Path=Production}"/>
                            <DataGridTextColumn Width="*" Header="Итоговая сум-
ма" Binding="{Binding Path=TotalPrice, StringFormat={}{0} ₽}"/>
                        </DataGrid.Columns>
                        <DataGrid.ContextMenu>
                            <ContextMenu>
                                <MenuItem Header="Добавить"
Click="AddProductSale"/>
                                <MenuItem Header="Редактировать"
Click="RedactProductSale"/>
                                <MenuItem Header="Удалить"
Click="DeleteProductSale"/>
                            </ContextMenu>
                        </DataGrid.ContextMenu>
                    </DataGrid>
                     </Grid>
```

С целью сокращения трудоемкости менеджеров на формирование отчетов добавим возможность выгрузки отчетов в формате Excel из приложения. Процесс интеграции данного модуля в приложение начинается с написания кода для создания кнопки. Далее проектируем специализированный класс для взаимодействия с документом Excel. Внутри окна описываем метод выгрузки.

Код функционала выгрузки данных:

```
private void ToExcel(object sender, RoutedEventArgs e)
{
    if (sender is MenuItem menuItem &&
int.TryParse(menuItem.CommandParameter.ToString(), out int param))
    {
        if (param == 2)
        {
            if (Partners_LB.SelectedIndex == -1)
            {
                MessageBox.Show("Рассматриваемый партнер отстутствует");
                return;
            }
            if (ProductSales_Table.Items.Count == 0)
            {
                MessageBox.Show("Рассматриваемый партнер не имеет истории поку-
пок");
                return;
```

```
            }
        }
        if (param == 0)
        {
            if (ProductSales_Table.Items.Count == 0)
            {
                if (Partners_LB.SelectedIndex == -1)
                    MessageBox.Show("Рассматриваемый партнер отстутствует, и
следовательно не имеет истории покупок", "Предупреждение");
                else
                    MessageBox.Show("Рассматриваемый партнер не имеет истории
покупок", "Предупреждение");
            }
        }
        List<PartnerDto> partnerDtos = [];
        foreach (PartnerDto partnerDto in Partners_LB.Items)
            partnerDtos.Add(partnerDto);
        List<ProductSalesDto> salesDtos = [];
        foreach (ProductSalesDto salesDto in ProductSales_Table.Items)
            salesDtos.Add(salesDto);
        TableReport ExcelReport = new()
        {
            partnerDtos = partnerDtos,
            salesDtos = salesDtos
        };
        var reportExcel = MarketExcelGenerator.Generate(ExcelReport, param);

        var saveFileDialog = new SaveFileDialog
        {
            Filter = "Excel файлы (*.xlsx)|*.xlsx"
        };

        if (saveFileDialog.ShowDialog() == true)
        {
            try
            {
                using var fs = new FileStream(saveFileDialog.FileName, File-
Mode.Create);
                using var writer = new BinaryWriter(fs);
                writer.Write(reportExcel);
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message);
                return;
            }
        }
    }
    ReWrite();
     }


using System;
using System.Collections.Generic;
using OfficeOpenXml;
using ClassLibrary.DTOs;

namespace ClassLibrary.Classes
{
    public class MarketExcelGenerator
    {
        public static byte[] Generate(TableReport tableReport, int isInOneFile)
```

```
        {
            ExcelPackage.License.SetNonCommercialPersonal("Aksenov");
            var package = new ExcelPackage();

            switch (isInOneFile)
            {
                case 0:
                case 1:
                    // Создаем таблицу для партнеров
                    CreatePartnerSheet(package, tableReport.partnerDtos);
                    // Создаем таблицу для истории продаж, если нужен
                    if (isInOneFile == 0 || isInOneFile == 2)
                    {
                        CreateSalesSheet(package, tableReport.salesDtos);
                    }
                    break;

                case 2:
                    CreatePartnerSheet(package, tableReport.partnerDtos);
                    CreateSalesSheet(package, tableReport.salesDtos);
                    break;
            }

            return package.GetAsByteArray();
        }

        // Метод для создания таблицы партнеров
        private static void CreatePartnerSheet(ExcelPackage package,
List<PartnerDto> partnerDTOs)
        {
            var worksheet = package.Workbook.Worksheets.Add("Партнеры");
            // Заголовки таблицы
            worksheet.Cells["B2"].Value = "Тип";
            worksheet.Cells["C2"].Value = "Наименование";
            worksheet.Cells["D2"].Value = "Скидка";
            worksheet.Cells["E2"].Value = "ФИО директора";
            worksheet.Cells["F2"].Value = "Телефон";
            worksheet.Cells["G2"].Value = "Рейтинг";

            int row = 3;
            foreach (var partner in partnerDTOs)
            {
                worksheet.Cells[$"B{row}"].Value = partner.Type;
                worksheet.Cells[$"C{row}"].Value = partner.Name;
                worksheet.Cells[$"D{row}"].Value = partner.Discount;
                worksheet.Cells[$"E{row}"].Value = partner.DirectorName;
                worksheet.Cells[$"F{row}"].Value = partner.Phone;
                worksheet.Cells[$"G{row}"].Value = partner.Rating;
                row++;
            }
        }

        // Метод для создания таблицы истории продаж
        private static void CreateSalesSheet(ExcelPackage package,
List<ProductSalesDto> salesDTOs)
        {
            var worksheet = package.Workbook.Worksheets.Add("История закупок");
            // Заголовки таблицы
            worksheet.Cells["B2"].Value = "Дата";
            worksheet.Cells["C2"].Value = "Количество";
            worksheet.Cells["D2"].Value = "Продукция";
            worksheet.Cells["E2"].Value = "Итоговая сумма";
```

```
            int row = 3;
            foreach (var sale in salesDTOs)
            {
                worksheet.Cells[$"B{row}"].Value = sale.Date;
                worksheet.Cells[$"C{row}"].Value = sale.Count;
                worksheet.Cells[$"D{row}"].Value = sale.Production;
                worksheet.Cells[$"E{row}"].Value = sale.TotalPrice;
                row++;
            }
        }

        public static byte[] GenerateBackUp(RootObject rootObject)
        {
            ExcelPackage.License.SetNonCommercialPersonal("Aksenov");
            var package = new ExcelPackage();

            // Создание таблицы для партнеров
            CreateBackupPartnerSheet(package, rootObject.Partners);
            // Создание таблицы для производств
            CreateBackupProductionSheet(package, rootObject.Productions);
            // Создание таблицы для продаж товаров
            CreateBackupSalesSheet(package, rootObject.Products);

            return package.GetAsByteArray();
        }

        private static void CreateBackupPartnerSheet(ExcelPackage package,
List<Partner> partners)
        {
            var sheet = package.Workbook.Worksheets.Add("Партнеры");
            sheet.Cells["B2"].Value = "Id";
            sheet.Cells["C2"].Value = "Тип";
            sheet.Cells["D2"].Value = "Наименование";
            sheet.Cells["E2"].Value = "Юридический адрес";
            sheet.Cells["F2"].Value = "ИНН";
            sheet.Cells["G2"].Value = "ФИО директора";
            sheet.Cells["H2"].Value = "Телефон";
            sheet.Cells["I2"].Value = "Email";
            sheet.Cells["J2"].Value = "Логотип";
            sheet.Cells["K2"].Value = "Рейтинг";

            int row = 3;
            foreach (var partner in partners)
            {
                sheet.Cells[$"B{row}"].Value = partner.Id;
                sheet.Cells[$"C{row}"].Value = partner.Type;
                sheet.Cells[$"D{row}"].Value = partner.Name;
                sheet.Cells[$"E{row}"].Value = partner.LegalAddress;
                sheet.Cells[$"F{row}"].Value = partner.Inn;
                sheet.Cells[$"G{row}"].Value = partner.DirectorName;
                sheet.Cells[$"H{row}"].Value = partner.Phone;
                sheet.Cells[$"I{row}"].Value = partner.Email;
                sheet.Cells[$"J{row}"].Value = partner.Logo;
                sheet.Cells[$"K{row}"].Value = partner.Rating;
                row++;
            }
        }

        private static void CreateBackupProductionSheet(ExcelPackage package,
List<Production> productions)
        {
```

```
            var sheet = package.Workbook.Worksheets.Add("Производство");
            sheet.Cells["B2"].Value = "Id";
            sheet.Cells["C2"].Value = "Артикул";
            sheet.Cells["D2"].Value = "Наименование";
            sheet.Cells["E2"].Value = "Описание";
            sheet.Cells["F2"].Value = "Фото";
            sheet.Cells["G2"].Value = "Мини цена партнера";
            sheet.Cells["H2"].Value = "Длина";
            sheet.Cells["I2"].Value = "Ширина";
            sheet.Cells["J2"].Value = "Высота";
            sheet.Cells["K2"].Value = "Вес.net";
            sheet.Cells["L2"].Value = "Вес.брутт";
            sheet.Cells["M2"].Value = "Сертификат качества";
            sheet.Cells["N2"].Value = "Стандартный номер";
            sheet.Cells["O2"].Value = "Цена себестоимости";

            int row = 3;
            foreach (var prod in productions)
            {
                sheet.Cells[$"B{row}"].Value = prod.Id;
                sheet.Cells[$"C{row}"].Value = prod.Article;
                sheet.Cells[$"D{row}"].Value = prod.Name;
                sheet.Cells[$"E{row}"].Value = prod.Description;
                sheet.Cells[$"F{row}"].Value = prod.Photo;
                sheet.Cells[$"G{row}"].Value = prod.MinPartnerPrice;
                sheet.Cells[$"H{row}"].Value = prod.Length;
                sheet.Cells[$"I{row}"].Value = prod.Width;
                sheet.Cells[$"J{row}"].Value = prod.Height;
                sheet.Cells[$"K{row}"].Value = prod.NetWeight;
                sheet.Cells[$"L{row}"].Value = prod.GrossWeight;
                sheet.Cells[$"M{row}"].Value = prod.QualityCertificate;
                sheet.Cells[$"N{row}"].Value = prod.StandardNumber;
                sheet.Cells[$"O{row}"].Value = prod.CostPrice;
                row++;
            }
        }

        private static void CreateBackupSalesSheet(ExcelPackage package,
List<ProductSale> sales)
        {
            var sheet = package.Workbook.Worksheets.Add("Продажи товаров");
            sheet.Cells["B2"].Value = "Id";
            sheet.Cells["C2"].Value = "Partner Id";
            sheet.Cells["D2"].Value = "Production Id";
            sheet.Cells["E2"].Value = "Кол-во";
            sheet.Cells["F2"].Value = "Дата";
            sheet.Cells["G2"].Value = "Общая цена";

            int row = 3;
            foreach (var sale in sales)
            {
                sheet.Cells[$"B{row}"].Value = sale.Id;
                sheet.Cells[$"C{row}"].Value = sale.PartnerId;
                sheet.Cells[$"D{row}"].Value = sale.ProductionId;
                sheet.Cells[$"E{row}"].Value = sale.Count;
                sheet.Cells[$"F{row}"].Value = sale.SaleDate.ToString(); //
Можно оставить так или применить форматирование
                sheet.Cells[$"G{row}"].Value = sale.TotalPrice;
                row++;
            }
        }
    }
```

```
// Остальные классы данных - без изменений, их структура выглядит логичной.
public class RootObject
{
    public List<Partner> Partners { get; set; }
    public List<Production> Productions { get; set; }
    public List<ProductSale> Products { get; set; }
}

public class TableReport
{
    public List<PartnerDto> partnerDtos { get; set; }
    public List<ProductSalesDto> salesDtos { get; set; }
}
 }
```

3.1.8. Верификация и аттестация программного обеспечения

Верификацией и аттестацией называют процессы проверки и анализа, в ходе которых проверяется соответствие программного обеспечения своей спецификации и требованиям заказчиков. Верификация и аттестация охватывают полный жизненный цикл программного обеспечения – они начинаются на этапе анализа требований и завершаются проверкой программного кода на этапе тестирования готовой программной системы.

Верификация в большей степени направлена на внутренние процессы и осуществляется преимущественно разработчиками и QA-инженерами. Верификация проверяет соответствие программного обеспечения системной спецификации, в частности функциональным и нефункциональным требованиям.

Аттестация проводится после верификации, чтобы определить, насколько система соответствует не только спецификации, но и ожиданиям заказчика. На ранних этапах разработки программного обеспечения очень важна аттестация системных требований. В требованиях часто встречаются ошибки и упущения. В таких случаях конечный продукт, вероятно, не будет соответствовать ожиданиям заказчика. Но, аттестация требований не может выявить все проблемы в спецификации требований. Иногда недоработки и ошибки в требованиях обнаруживаются только после завершения реализации системы.

Главная цель верификации и аттестации — удостовериться в том, что система «соответствует своему назначению». Уровень необходимой достоверности соответствия зависит от назначения системы, ожиданий пользователей и условий на рынке программных продуктов.

В процессах верификации и аттестации используются две основные методики проверки и анализа систем:

1. Инспектирование программного обеспечения. Анализ и проверка различных представлений системы, например документации спецификации требований,

архитектурных схем или исходного кода программ. Инспектирование выполняется на всех этапах процесса разработки программной системы. Параллельно с инспектированием может выполняться автоматический анализ исходного кода программ и соответствующих документов. Инспектирование и автоматический анализ – это статические методы верификации и аттестации, поскольку им не требуется исполняемая система.

К методам инспектирования относятся: инспектирование программ, автоматический анализ исходного кода и формальная верификация. Но статические методы могут проверить только соответствие программ спецификации, с их помощью невозможно проверить правильность функционирования системы. Кроме того, статическими методами нельзя проверить такие нефункциональные характеристики, как производительность и надежность. Поэтому для оценивания нефункциональных характеристик проводится тестирование системы.

2. Тестирование программного обеспечения. Запуск исполняемого кода с тестовыми данными и исследование выходных данных и рабочих характеристик программного продукта для проверки правильности работы системы. Тестирование – это динамический метод верификации и аттестации, так как применяется к исполняемой системе.

Тестирование – это проверка работы программ с данными, подобными реальным, которые будут обрабатываться в процессе эксплуатации системы. Наличие в программе дефектов и несоответствий требованиям обнаруживается путем исследования выходных данных и выявления среди них аномальных. Тестирование выполняется на этапе реализации системы (для проверки соответствия системы ожиданиям разработчиков) и после завершения ее реализации.

На разных этапах процесса разработки программного обеспечения применяют различные виды тестирования.

1. Тестирование дефектов проводится для обнаружения несоответствий между программой и ее спецификацией, которые обусловлены ошибками или дефектами в программах. Такие тесты разрабатываются для выявления ошибок в системе, а не для имитации ее работы.

2. Статистическое тестирование оценивает производительность и надежность программ, а также работу системы в различных режимах эксплуатации. Тесты разрабатываются так, чтобы имитировать реальную работу системы с реальными входными данными. Надежность функционирования системы оценивается по количеству сбоев, отмеченных в работе программ. Производительность оценивается по результатам измерения полного времени выполнения операций и времени отклика системы при обработке тестовых данных.

Как правило, в ходе верификации и аттестации в системе обнаруживаются ошибки. Для исправления ошибок в систему вносятся изменения. Этот процесс отладки обычно интегрирован с другими процессами верификации и аттестации. Вместе с тем тестирование (или более обобщенно – верификация и аттестация) и отладка являются разными процессами, которые имеют различные цели.

Обнаруженная в программе ошибка исправляется, после чего необходимо снова проверить программу. Для этого можно еще раз выполнить инспектирование программы или повторить предыдущее тестирование. Повторное тестирование используется для того, чтобы убедиться, что сделанные в программе изменения не внесли в систему новых ошибок, поскольку на практике высокий процент "исправления ошибок" либо не завершается полностью, либо вносит новые ошибки в программу.

В принципе во время повторного тестирования после каждого исправления необходимо еще раз запускать все тесты, однако на практике такой подход оказывается слишком дорогостоящим. Поэтому при планировании процесса тестирования определяются зависимости между частями системы и назначаются тесты для каждой части. Тогда можно трассировать программные элементы с помощью специальных контрольных примеров (контрольных данных), подобранных для этих элементов. Если результаты трассировки задокументированы, то для проверки измененного программного элемента и зависимых от него компонентов можно использовать только некоторое подмножество всего множества тестовых данных.

Верификация и аттестация – дорогостоящий процесс. Для больших систем, половина бюджета, выделенного на разработку системы, тратится на процесс верификации и аттестации.

ЗАКЛЮЧЕНИЕ

В результате прохождения производственной практики с 09 по 17 марта 2026 г. были закреплены теоретические знания и расширены профессиональные умения.

По итогам практики получены следующие результаты:

1. Изучена IT-инфраструктура ПГГПУ и роль программного обеспечения в его бизнес-процессах.
2. Разработано программное обеспечение.
3. Осуществлено инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования.
4. Выполнена отладка программного модуля с использованием специализированных программных средств.
5. Реализована разработка тестовых наборов и тестовых сценариев для программного обеспечения.
6. Выполнена интеграция модулей в программное обеспечение.
7. Осуществлена верификация и аттестация программного обеспечения.

По окончании практики был получен практический опыт интеграции модулей в программное обеспечение; отладки программного модуля с использованием специализированных программных средств; разработки тестовых наборов и тестовых сценариев для программного обеспечения; инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. ГОСТ 19.201-78* - Единая система программной документации. Техническое задание. Требования к содержанию и оформлению.

2. ГОСТ 34.602-2020 - Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы.

3. Федорова Г. Н. Участие в интеграции программных модулей. Учебное пособие / Федорова Г.Н. – М.Ж Издательский дом «Академия», 2016 – 304с.

4. Федорова Г. Н. Разработка, внедрение и адаптация программного обеспечения отраслевой направленности: Учебное пособие. / Федорова Г.Н. - М.:КУРС, НИЦ ИНФРА-М, 2016. - 336 с. http://znanium.com/catalog.php?bookinfo=544732

5. Оценка качества программного обеспечения: Практикум: Учебное пособие / Б.В. Черников, Б.Е. Поклонов; Под ред. Б.В. Черникова - М.: ИД ФОРУМ: НИЦ Инфра-М, 2012. - 400 с. http://znanium.com/bookread2.php?book=315269

6. Гуриков С. Р. Введение в программирование на языке Visual C#: Учебное пособие / С.Р. Гуриков. - М.: Форум: НИЦ ИНФРА-М, 2013. - 448 с. http://znanium.com/bookread2.php?book=232661

7. Исаченко О.В. Программное обеспечение компьютерных сетей : учебное пособие. М. : ИНФРА-М, 2013. - 117 с. - (СПО).

8. Стандартизация, сертификация и управление качеством программного обеспечения: Учебное пособие / Ананьева Т.Н., Новикова Н.Г., Исаев Г.Н. - М.:НИЦ ИНФРА-М, 2016. - 232с http://znanium.com/bookread2.php?book=541003

9. Гагарина Л.Г. Технология разработки программного обеспечения: Учебное пособие / Л.Г. Гагарина, Б.Д. Виснадул, Е.В. Кокорева. М.: ООО "Научно-издательский центр ИНФРА-М", 2013. 400 с. URL: http://znanium.com/catalog.php?bookinfo=389963

10. Немцова Т. И. Программирование на языке высокого уровня. Программир. на языке С++[Электронный ресурс]: Уч. пос. / Т.И.Немцова и др.; Под ред. Л.Г.Гагариной. М.: ИД ФОРУМ: ИНФРА-М, 2012. 512 с. URL: http://znanium.com/catalog.php?bookinfo=244875;

11. Рудаков А.В. Технология разработки программных продуктов: Учеб.пособие для студ. сред. проф. образования. М.: Издательство «Академия», 2010. 208 с.

12. Черников Б.В. Управление качеством программного обеспечения [Электронный ресурс]: Учебник / Б.В. Черников. - М.: ИД ФОРУМ: ИНФРА-М, 2012. - 240 с.. URL: http://znanium.com/catalog.php?bookinfo=256901.

13. Эйдлина Г.М. Delpfi: программирование в примерах и задачах[Электронный ресурс]:

Практикум / Г.М. Эйдлина, К.А. Милорадов. М.: ИЦ РИОР: НИЦ Инфра-М, 2012. 116 с. URL: http://znanium.com/catalog.php?bookinfo=319046

14. Технология программирования: [Электронныйресурс]. URL: http://glossary.ru/cgi-bin/gl_sch2.cgi?RSl)turujoo!vwujwgssowuigto9

15. Технология программирования:[Электронный ресурс].URL: http://2 programmer. ru/l1?start=3

16. Языки программирования: [Электронный ресурс]. URL: http://maksakov-sa.ru/TehProgram/index.html

17. Базы данных: [Электронный ресурс]. URL: http://www.sdteam.com/5/